

Amman Khawaja

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
```

```
  var i:int := a.Length-1;
  out := 0;
  while (i >= 0)
    invariant i >= 0
  {
    out := out - a[i];
    i := i-1;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
```

```
  var i:int := 0;
  out := 0;
  while (i < a.Length)
    invariant i <= a.Length
  {
    out := out - a[i];
    i := i+1;
  }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x y \rightarrow$ if $x == y$ then show x else show (x,y)

$\text{String} \rightarrow \text{String} \rightarrow \text{IO String}$

(d) $\backslash x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) $\text{foldr} (\text{const } l)$

$b \rightarrow [a] \rightarrow b$

(f) $(^* 42^*)$

ill-typed

(g) $(.) ^* 42^*$

$[a] \rightarrow [a]$

(h) $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a] \rightarrow [a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter} (< 42) (l ++ l)$

$[Int] \rightarrow [Int]$

(j) $\text{let } f \ x = x \text{ in } (f 'a' f \text{ True})$

ill-typed

(k) $\text{fmap} (\text{fmap} (+1)) [\text{Nothing}]$

ill-typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

Example answers:

`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`

`\x -> [x]`

(c) `a -> Maybe b`

`\x -> if a == 0 then Nothing else Maybe True`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`\f i c ->`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`\f Maybe a Maybe b -> Maybe f a b`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`\a b c -> foldr (\x acc -> if b (a x) then (a x).acc) [] c`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

`\a f -> if a /= Nothing then`

(h) `Eq a => a -> [a] -> [a]`

`\a b -> foldr (\x acc -> if a == x then a.acc) [] b`

(i) `Show a => [a] -> IO String`

`\(a:at) -> show a`

(j) `(a,b) -> (a -> b -> c) -> c`

`\(a,b) f -> f a b`

(k) `((a,b) -> c) -> c`

`a`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

$m=2$
 $x=4$
 $z=2$
 $\text{add } m^2, m \text{ times}$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules — write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow>$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } }  $\rightarrow>$ 
{ { m > 0 } }  $\wedge$   $0 = m \cdot m \cdot (m - m)$ 
  y := 0;
{ { m > 0 } }  $\wedge$  y = m * m * (m - m)
  var x := m * m;
{ { m > 0 } }  $\wedge$  y = x * (m - m)
  var z := m;
{ { z > 0 } }  $\wedge$  y = x * (m - z)
  while (z > 0) {
    { { z > 0 } }  $\wedge$  y = x * (m - z)
    { { z > 0 } }  $\wedge$  y + x = x * (m - z - 1)
    z := z - 1;
    { { z > 0 } }  $\wedge$  y + x = x * (m - z)
    y := y + x;
    { { z > 0 } }  $\wedge$  y = x * (m - z)
  }
{ { z == 0 } }  $\wedge$  y = x * (m - z)
{ { y = m * m * m } }
```

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave a b =`
 `where`
 `whelp a b [] = []`
 `whelp (a:at) (b:bt) out = whelp at b+ (out++[a]++[b])`
 `whelp _ _ out = out`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[], [1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`powerset list =`
 `powersetHelp list [] where`
 `powersetHelp (a:at) out = powersetHelp at (a:out)`

Cole Thakill
UID - 117145403

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

[1,2,3] 7

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
```

```
method f2(a:array<int>) returns (out:int) {
```

10/14

```
out := 0;
var i := 0;
while i < a.length {
  out := out - a[i];
  i := i + 1
}
```

```
out := 0;
var i := a.length - 1;
while i >= 0 {
  out := out - a[i];
  i := i - 1
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x\ y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x\ y \rightarrow$ if $x == y$ then show x else show (x,y) ✖

$a \rightarrow a \rightarrow \text{String}$

(d) $\backslash x\ l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) `foldr (const 1)`

Ill-typed

(f) `(,"42")`

Ill-typed

(g) `(,) "42"`

$a \rightarrow (\text{String}, a)$

(h) `reverse . reverse`

Ill-typed

(i) $\backslash l \rightarrow$ filter (< 42) $(l ++ l)$

$[\text{Int}] \rightarrow [\text{Int}]$

(j) let $f\ x = x$ in $(f\ 'a', f\ \text{True})$

~~**Ill-typed**~~ **Ill-typed**

(k) `fmap (fmap (+1)) [Nothing]`

Ill-typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) $\text{Int} \rightarrow \text{Int}$

Example answers:

$\backslash x \rightarrow x + 1$

$(+1)$

(b) $\text{Bool} \rightarrow [\text{Bool}]$

$\backslash x \rightarrow \text{if } x \text{ then } [\text{True}] \text{ else } [\text{False}]$

(c) $a \rightarrow \text{Maybe } b$

$\backslash x \rightarrow \text{Nothing}$

(d) $(\text{Int} \rightarrow \text{Char} \rightarrow \text{Bool}) \rightarrow [\text{Int}] \rightarrow [\text{Char}] \rightarrow [\text{Bool}]$

undefined

(e) $(a \rightarrow b \rightarrow c) \rightarrow \text{Maybe } a \rightarrow \text{Maybe } b \rightarrow \text{Maybe } c$

undefined

(f) $(a \rightarrow b) \rightarrow (b \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [b]$

undefined

(g) $\text{Maybe } a \rightarrow (a \rightarrow [b]) \rightarrow [(a, b)]$

undefined

(h) $\text{Eq } a \Rightarrow a \rightarrow [a] \rightarrow [a]$

$\backslash x \rightarrow \text{if } x == (\text{head } []) \text{ then } [x] \text{ else } []$

(i) $\text{Show } a \Rightarrow [a] \rightarrow \text{IO String}$

$\backslash a \rightarrow \text{putStrLn } (\text{show } a)$

(j) $(a, b) \rightarrow (a \rightarrow b \rightarrow c) \rightarrow c$

$\backslash (x, y) \rightarrow f \times y$

(k) $((a, b) \rightarrow c) \rightarrow c$

undefined

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (v:int)
  requires m > 0
  ensures v == m*m*m
{
  v := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    v := v + x;
  }
}
```

Handwritten annotations for the first iteration (z=4):

y	0	→	4	16
x	16		x	16
z	4		z	3

Handwritten annotations for the second iteration (z=3):

y	16	→	4	32
x	16		x	16
z	3		z	2

Handwritten annotations for the third iteration (z=2):

y	32	→	4	48
x	16		x	16
z	2		z	1

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post-conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules - write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow$. These implication steps can (silently) rely on all the usual rules of arithmetic.

$z \leq m$

```
{ { m > 0 } }  $\rightarrow$ 
{ { 0 = m * m * (m - m) } }
  y := 0;
  { { y = m * m * (m - m) } }
    var x := m * m;
    var z := m;
    { { y = x * (m - 2) } }
    while (z > 0) {
      { { z > 0 && y = x * (m - 2) } }
      { { y + x = x * (m - 2 + 1) } }
      z := z - 1;
      { { y + x = x * (m - 2) } }
      y := y + x;
      { { y = x * (m - 2) } }
    }
    { { !(z > 0) && y = x * (m - 2) } }
  { { y = m * m * m } }  $\rightarrow$ 
```

Beginning

y	0		End	y	m * m * m
x	m * m			x	m * m
z	m			z	0

Handwritten annotations for the beginning state:

y	0		48
x	16		16
z	4		4

Handwritten annotations for the end state:

y	0	16	32	48	64
z	4	3	2	1	0

x constant at 16

Invariant $\rightarrow$ $y = x * (m - z)$

Handwritten calculations for the invariant:

- $0 = 16 * (4 - 4)$
- $16 = 16 * (4 - 3)$
- $32 = 16 * (4 - 2)$
- $48 = 16 * (4 - 1)$
- $64 = 16 * (4 - 0)$

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave :: [a] -> [a] -> [a]`
`weave [] = []`
`weave _ [] = []`
`weave (x:xs) (y:ys) = x:[y]:(weave xs ys)`

two folds?

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`powerset :: [a] -> [[a]]`

~~`powerset :: [a] -> [[a]]`~~

`powerset [] = [[]]`

`powerset xs =`

Alexandra Yates
117830408

CMSC 433: Midterm Exam (Spring 2024)

f1

Question 1 (20 points)

1 2 3 4

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  requires a.Length > 0
```

```
method f2(a:array<int>) returns (out:int) {
  requires a.Length > 0
```

```
  out := 0;
  var i := 0;
  while (i < a.Length)
    invariant 0 ≤ i ≤ a.Length
  {
    out := out - a[i];
    i := i + 1;
  }
  return out;
}
```

```
  out := 0;
  var i := a.Length - 1;
  while (i ≥ 0)
    invariant a.Length ≤ i ≤ -1
  {
    out := out - a[i];
    i := i - 1;
  }
  return out;
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x y \rightarrow (x, y)$

$a \rightarrow b \rightarrow (a, b)$

(c) $\backslash x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x, y)$

ill-typed

(d) $\backslash x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) $\text{foldr } (\text{const } 1)$

$(a \rightarrow b \rightarrow a) \rightarrow a \rightarrow b \rightarrow a$

(f) ("42")

(a, string)

(g) ("42")

ill-typed

(h) $\text{reverse}, \text{reverse}$

Too many args

ill-typed

(i) $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$[Int] \rightarrow [Int]$

(j) $\text{let } f x = x \text{ in } (f 'a', f \text{True})$

ill-typed

(k) $\text{fmap } (\text{fmap } (+1)) [\text{Nothing}]$

ill-typed

Question 3 (20 points)

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules — write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow>$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } }  $\rightarrow>$ 
m * m == m = m * m * m 44 m > 0
  y := 0;
  m * m * m = m * m * m 44 m > 0
    var x := m * m;
    m * x = m * m * m 44 m > 0
      var z := m;
      { { 2 * x == m * m * m 44 z > 0
        while (z > 0) {
          2 * x == m * m * m 44 z > 0
          { { 2 * x == m * m * m 44 z > 0
            z := z - 1;
            (2 * 1) * x = m * m * m 44 z > 0
              y := y + x;
              { { 2 * x == m * m * m
            }
          } { { 2 * x == m * m * m 44 ! (z > 0)
            { { y = m * m * m * m } }
          }  $\rightarrow>$ 
```


Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave [] [] = []`

`weave (x:xs) (y:ys) = x:y:weave xs ys`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3]]`

`powerset [] = [[]]`

`[] = [[]]`

`(x:xs) =-> 1 : x : helper x xs powerset xs` where

`helper a as = case as of`

`[] => []`

`(y:ys) => [a,y] : helper a ys`

Garrett Beebe
118015873

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
```

```
  out := 0;
```

```
  var i : int := a.Length;
```

```
  while (i > 0) {
```

```
    out := out - a[i];
```

```
    i := i - 1;
```

```
  }
```

```
method f2(a:array<int>) returns (out:int) {
```

```
  out := 0;
```

```
  var i : int := 0;
```

```
  while (i < a.Length) {
```

```
    out := out - a[i];
```

```
    i := i + 1;
```

```
  }
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\lambda x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\lambda x y \rightarrow (x, y)$

$a \rightarrow b \rightarrow (a, b)$

(c) $\lambda x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x, y)$

$a \rightarrow a \rightarrow \text{String}$

(d) $\lambda x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) $\text{foldr } (\text{const } 1)$

undefined

(f) $(\cdot "42")$

undefined

(g) $(\cdot "42")$

undefined

(h) $\text{reverse} . \text{reverse}$

undefined

(i) $\lambda l \rightarrow \text{filter } (< 42) (l ++ l)$

$[Int] \rightarrow [Int]$

(j) $\text{let } f \ x = x \text{ in } (f 'a', f \text{ True})$

$(a \rightarrow b) \rightarrow c \rightarrow (b, b)$

(k) $\text{fmap } (\text{fmap } (+1)) [\text{Nothing}]$

undefined

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) $\text{Int} \rightarrow \text{Int}$

Example answers:

$\backslash x \rightarrow x + 1$
(+1)

(b) $\text{Bool} \rightarrow [\text{Bool}]$

$\backslash x \rightarrow \text{case } x \text{ of } \text{True} \rightarrow [x] \mid \text{False} \rightarrow []$

(c) $a \rightarrow \text{Maybe } b$

$\backslash x \rightarrow \text{Nothing}$

(d) $(\text{Int} \rightarrow \text{Char} \rightarrow \text{Bool}) \rightarrow [\text{Int}] \rightarrow [\text{Char}] \rightarrow [\text{Bool}]$

$\backslash f \ x \ y \rightarrow \text{zipWith } f \ [(1..x)] \ [(1..y)]$

(e) $(a \rightarrow b \rightarrow c) \rightarrow \text{Maybe } a \rightarrow \text{Maybe } b \rightarrow \text{Maybe } c$

$\backslash f \ x \ y \rightarrow \text{Just } (f \ x \ y)$

(f) $(a \rightarrow b) \rightarrow (b \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [b]$

$\backslash f \ p \ (x:xs) \rightarrow \text{if } p \ (f \ x) == \text{True} \text{ then } [f \ x]$

(g) $\text{Maybe } a \rightarrow (a \rightarrow [b]) \rightarrow [(a,b)]$

$\backslash a \ f \rightarrow \text{case } a \text{ of}$

(h) $\text{Eq } a \Rightarrow a \rightarrow [a] \rightarrow [a]$

$\backslash a \ (x:xs) \rightarrow \text{if } a == x \text{ then } xs \text{ else } []$

(i) Show $a \Rightarrow [a] \rightarrow \text{IO String}$

$\backslash a \rightarrow \text{putStrLn } (a ++ a)$

(j) $(a,b) \rightarrow (a \rightarrow b \rightarrow c) \rightarrow c$

$\backslash (a,b) \ f \rightarrow f \ a \ b$

(k) $((a,b) \rightarrow c) \rightarrow c$

$\backslash f \rightarrow$

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence so that if you work backwards from the end of the program you should only rely on all the usual rules of arithmetic.

```
{ { m > 0 } } ->>
{ { 0 = 0
  y := 0;
  { { y = 0
    var x := m * m;
    { { x = m * m
      var z := m;
      { { x = m * m
        while (z > 0) {
          { { y = m * m * m
            { { y = m * m * m
              z := z - 1;
              { { y = m * m * m
                y := y + x;
                { { y = m * m * m
                  }
                  { { y = m * m * m
                    { { y = m * m * m
                      { { 1 (z > 0)
                        } } ->>
                      } }
                    } }
                  } }
                } }
              } }
            } }
          } }
        } }
      } }
    } }
  } }
}
```


Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave [] [] = []`

`weave - [] = []`

`weave [] - = []`

`weave (x:xs) (y:ys) = (x:[y]) ++ (weave xs ys)`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`powerset [] = [[]]`

`powerset [] = []`

`powerset x = powerset [x] []` where

`powerset - [] = []`

`powerset [] - = []`

`powerset`

Saipraneeth Muvva

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)
```

```
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0
```

```
f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

<pre>method f1(a:array<int>) returns (out:int) { var i:int := a.length-1; out := 0; while i >= 0 { out := out - a[i]; i := i-1; } }</pre>	<pre>method f2(a:array<int>) returns (out:int) { var i:int := 0; out := 0; while i < a.length { out := out + a[i]; i := i+1; } }</pre>
--	---

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x\ y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a, b)$

(c) $\backslash x\ y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

$a \rightarrow b \rightarrow (a, b)$

(d) $\backslash x\ l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) `foldr (const 1)`

$(a \rightarrow b \rightarrow a) \rightarrow b \rightarrow a \rightarrow b$

(f) `("42")`

$a \rightarrow \text{String} \rightarrow (a, \text{String})$

(g) `(.) "42"`

$a \rightarrow \text{String} \rightarrow (a, \text{String})$

(h) `reverse . reverse`

$([b] \rightarrow [c]) \rightarrow ([a] \rightarrow [b]) \rightarrow a \rightarrow c$

(i) $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$[a] \rightarrow (a \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [a]$

(j) `let f x = x in (f 'a', f True)`

$(a \rightarrow b) \rightarrow b$

(k) `fmap (fmap (+1)) [Nothing]`

$a \rightarrow (a \rightarrow b) \rightarrow a \rightarrow \text{Maybe } a$

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) $\text{Int} \rightarrow \text{Int}$

Example answers:

$\backslash x \rightarrow x + 1$

$(+1)$

(b) $\text{Bool} \rightarrow [\text{Bool}]$

$\backslash x \rightarrow \text{if } x \text{ then } [] \mid x \text{ else } [x] \mid !x$

(c) $a \rightarrow \text{Maybe } b$

$\backslash x \rightarrow \text{Nothing}$

(d) $(\text{Int} \rightarrow \text{Char} \rightarrow \text{Bool}) \rightarrow [\text{Int}] \rightarrow [\text{Char}] \rightarrow [\text{Bool}]$

$\text{let } f \ x \ y = f(x+1) \ y$

(e) $(a \rightarrow b \rightarrow c) \rightarrow \text{Maybe } a \rightarrow \text{Maybe } b \rightarrow \text{Maybe } c$

$\text{let } f \ x \ y = \text{Just } (f \ \text{Just } x \ \text{Just } y)$

(f) $(a \rightarrow b) \rightarrow (b \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [b]$

$\text{let } f \ g \ l = \text{if } (g \ (f \ (fst \ l))) \text{ then } [f \ (fst \ l)] \text{ else } [f \ (fst \ l)]$

(g) $\text{Maybe } a \rightarrow (a \rightarrow [b]) \rightarrow [(a,b)]$

$\text{let } x \ f = (f \ (\text{Just } x), \text{fst } f(\text{Just } x))$

(h) $\text{Eq } a \Rightarrow a \rightarrow [a] \rightarrow [a]$

$\text{delete } x \ y$

(i) $\text{Show } a \Rightarrow [a] \rightarrow \text{IO String}$

(j) $(a,b) \rightarrow (a \rightarrow b \rightarrow c) \rightarrow c$

$\text{let } f \ g = g(a \ b)$

(k) $((a,b) \rightarrow c) \rightarrow c$

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and potentiaic in the way you apply Hoare rules - write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } }  $\rightarrow$ 
{ { 0 + (m * m) = m * m . (m - 1 - m) } }
  y := 0;
{ { y + (m * m) = m * m . (m - 1 - m) } }
  var x := m * m;
{ { y + x = x . (m - 1 - m) } }
  var z := m;
{ { y + x = x . (z - 1 - m) } }
  while (z > 0) {
    { { y + x = x . (z - 1 - m) } }
    { { y + x = x . (z - 1 - m) } }
    z := z - 1;
    { { y + x = x . (z - m) } }
    y := y + x;
    { { y = x . (z - m) } }
  }
{ { y = x . (z - m) } }
{ { y = m * m * m } }  $\rightarrow$ 
```

Handwritten annotations for the program:

$m = 3$

0	1	2	3	4	5	6
y	0	9	18	27		
x	9	9	9	9	9	9
z	3	2	1	0		

Result: 27

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
```

```
weave (x:xs) (y:ys) = (x:y) ++ weave xs ys
```

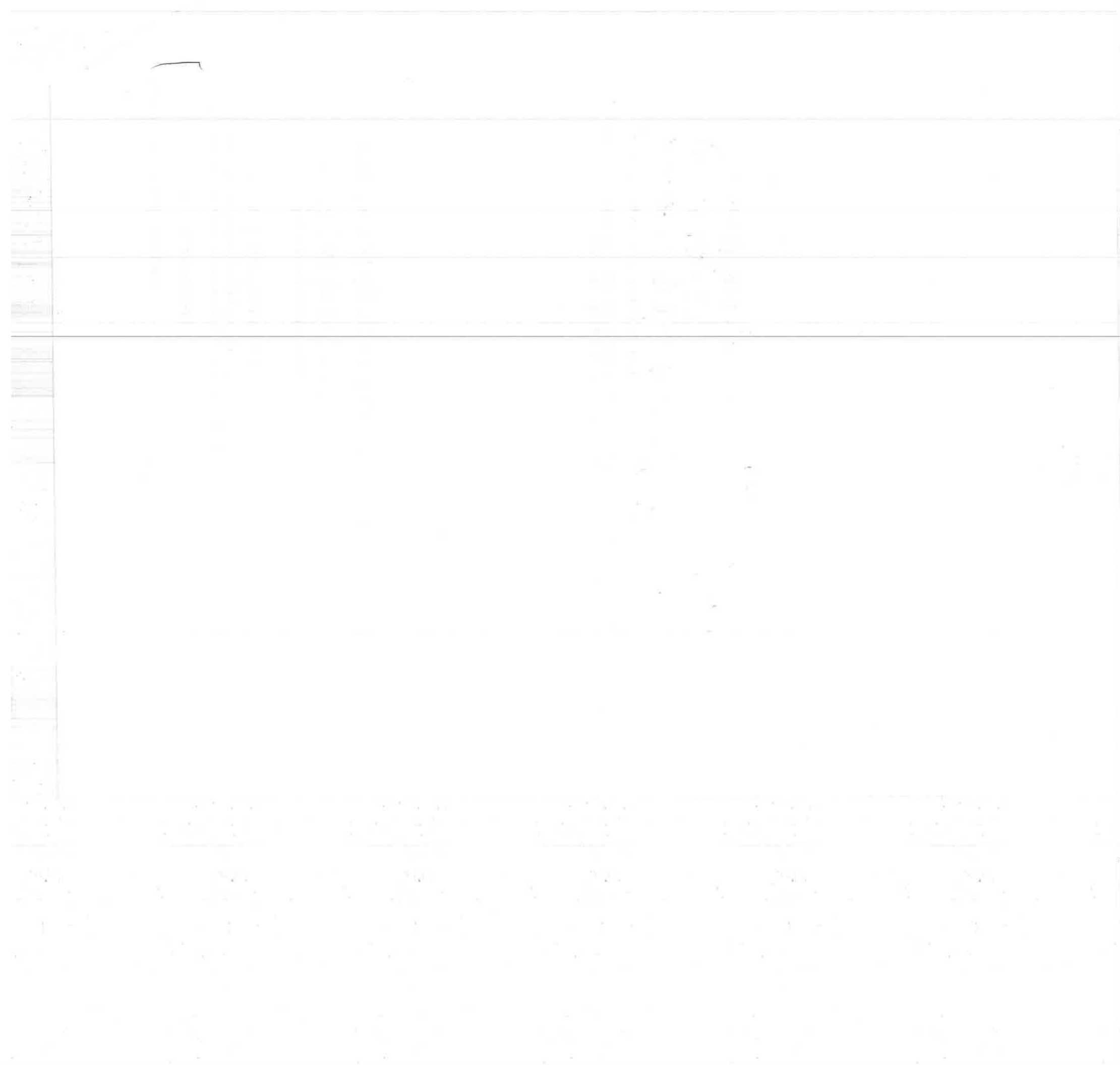
- (b) Implement a function `powerSet`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerSet [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]
```

```
powerSet :: [a] -> [[a]]
```

```
powerSet [] = []
```

```
powerSet (x:xs) = (x:xs) ++ powerSet xs
```



117792829

Zain Ashraf

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

<pre>method f1(a:array<int>) returns (out:int) { var i : int := a.Length - 1; out := 0; while (i >= 0) { out := out - a[i]; i := i - 1; } }</pre>	<pre>method f2(a:array<int>) returns (out:int) { var i : int := 0; out := 0; while (i < a.Length) { out := out - a[i]; i := i + 1; } }</pre>
--	---

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$
 $(a \rightarrow \text{String}) \rightarrow \text{String} \rightarrow \text{String} \rightarrow \text{String}$

(d) $\backslash x l \rightarrow x : l ++ l ++ [x]$

$\text{Int} \rightarrow [\text{Int}] \rightarrow [\text{Int}]$

(e) $\text{foldr} (\text{const } 1)$

$[a] \rightarrow a$

(f) $(^42)$

$!!1 - 1$

(g) $() ^42$

$(a \rightarrow b) \rightarrow a \rightarrow (a,b)$

(h) $\text{reverse} . \text{reverse}$

$([a] \rightarrow [a]) \rightarrow (b \rightarrow c) \rightarrow ([a] \rightarrow [c])$

(i) $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$a \rightarrow (a \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [a]$

(j) $\text{let } f \ x = x \text{ in } (f 'a', f \text{ True})$

$(a \rightarrow b) \rightarrow a \rightarrow b$

(k) $\text{fmap } (\text{fmap } (+1)) [\text{Nothing}]$

$(a \rightarrow b) \rightarrow (a \rightarrow b) \rightarrow a \rightarrow (a \rightarrow b) \rightarrow b$

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

Example answers:

`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`

`\True -> [True]`

(c) `a -> Maybe b`

`\a -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`\f x y -> x!y!([x y])`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`\a b c -> Maybe c`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`\f y a -> map y a`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

`\a -> undefined`

(h) `Eq a => a -> [a] -> [a]`

`\a -> [a]`

(i) `Show a => [a] -> IO String`

`\a -> Just a`

(j) `(a,b) -> (a -> b -> c) -> c`

`\a -> zipWith a`

(k) `((a,b) -> c) -> c`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

$$\begin{aligned} m &= 3 \\ x &= 4 \\ z &= 2 \end{aligned}$$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post-conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with `->>`. These implication steps can (silently) rely on all the usual rules of arithmetic.

```

{{ m > 0 }} -->
{{ z := 0; g8 0 := m.m.m }}
  y := 0;
  {{ z := 0; g8 y := m.m.m }}
    var x := m * m;
    {{ m := 0; g8 y := x.m }}
      var z := m;
      {{ z := 0; g8 y := x.m }}
        while (z > 0) {
          {{ z := 0; g8 y := x.m; g8 z := 0 }}
          {{ z - 1 > 0; g8 y + x := x.m }}
          z := z - 1;
          {{ z := 0; g8 y + x := x.m }}
            y := y + x;
            {{ z := 0; g8 y := x.m }}
              }
              {{ z := 0; g8 y := x.m; g8 ! (z := 0) }}
                {{ y = m * m * m }}
                -->

```

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave :: [a] -> [a] -> [a]`

`weave [] [] = []`

`weave (x:xs) (y:ys) = (x:y):weave xs ys`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3]]`

`powerset :: [a] -> [[a]]`

`powerset [] = [[]]`

`powerset (x:xs) = zipWith (:) (x:xs) . powerset xs`

Matthew Morrison

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i:int := a.length - 1;
  out := 0;
  while (i >= 0) {
    out := out - a[i];
    i := i - 1;
  }
}

method f2(a:array<int>) returns (out:int) {
  var i:int := 0;
  out := 0;
  while (i < a.length) {
    out := out + a[i];
    i := i + 1;
  }
}
```

$$f :: (a \rightarrow b, a) \rightarrow [a] \rightarrow [a]$$

$$rev :: [a] \rightarrow [a]$$

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$$a \rightarrow a$$

(b) $\backslash x y \rightarrow (x, y)$

$$a \rightarrow b \rightarrow (a, b)$$

(c) $\backslash x y \rightarrow$ if $x == y$ then show x else show (x, y)

$$a \rightarrow a \rightarrow \text{String}$$

(d) $\backslash x l \rightarrow x : l ++ l ++ [x]$

$$a \rightarrow [a] \rightarrow [a]$$

(e) $\text{foldr} (\text{const } 1)$

undefined

(f) $(^"42")$

$$a \rightarrow (a, \text{String})$$

(g) $(^"42")$

$$() \rightarrow ((), \text{String})$$

(h) $\text{reverse} . \text{reverse}$

$$([a] \rightarrow [a]) \rightarrow ([a] \rightarrow [a]) \rightarrow ([a] \rightarrow [a])$$

(i) $\backslash l \rightarrow \text{filter} (< 42) (l ++ l)$

$$[a] \rightarrow [a] \rightarrow [a]$$

(j) $\text{let } f x = x \text{ in } (f 'a', f \text{ True})$

undefined

(k) $\text{fmap} (\text{fmap} (+1)) [\text{Nothing}]$

$$(a \rightarrow a) \rightarrow [\text{map } a]$$

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) $\text{Int} \rightarrow \text{Int}$

Example answers:

$\backslash x \rightarrow x + 1$

$(+1)$

(b) $\text{Bool} \rightarrow [\text{Bool}]$

$\backslash x \rightarrow [x \text{ ~~is~~ true}]$

(c) $a \rightarrow \text{Maybe } b$

$\backslash x \rightarrow \text{!} \text{bool} \text{evg } x \text{ } [C1, true]$

(d) $(\text{Int} \rightarrow \text{Char} \rightarrow \text{Bool}) \rightarrow [\text{Int}] \rightarrow [\text{Char}] \rightarrow [\text{Bool}]$

undefined

(e) $(a \rightarrow b \rightarrow c) \rightarrow \text{Maybe } a \rightarrow \text{Maybe } b \rightarrow \text{Maybe } c$

undefined

(f) $(a \rightarrow b) \rightarrow (b \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [b]$

undefined

(g) $\text{Maybe } a \rightarrow (a \rightarrow [b]) \rightarrow [(a, b)]$

undefined

(h) $\text{Eq } a \Rightarrow a \rightarrow [a] \rightarrow [a]$

undefined

(i) $\text{Show } a \Rightarrow [a] \rightarrow \text{IO String}$

undefined

(j) $(a, b) \rightarrow (a \rightarrow b \rightarrow c) \rightarrow c$

undefined

(k) $((a, b) \rightarrow c) \rightarrow c$

undefined

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post-conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules – write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } }  $\rightarrow$  { { m > 0 } }  $\rightarrow$  { { m > 0 } }
{ { 0 < m < m } }  $\rightarrow$  { { m > 0 } }
{ { y := 0;
  { { (y + x) == m } }  $\rightarrow$  { { m > 0 } }
  var x := m * m;
  { { (y + x) == m } }  $\rightarrow$  { { m > 0 } }
  var z := m;
  { { (y + x) == m } }  $\rightarrow$  { { m > 0 } }
  while (z > 0) {
    { { (y + x) == m } }  $\rightarrow$  { { m > 0 } }
    { { (y + x) == m } }  $\rightarrow$  { { m > 0 } }
    z := z - 1;
    { { (y + x) == m } }  $\rightarrow$  { { m > 0 } }
    y := y + x;
    { { y == m } }  $\rightarrow$  { { m > 0 } }
  }
}
{ { y == m } }  $\rightarrow$  { { m > 0 } }
{ { y == m } }  $\rightarrow$  { { m > 0 } }
```


Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave a b =
  let z = Zip a b in
  weaveAux z where
    weaveAux ((i,j):r) =
      (i:j:weaveAux r)
    weaveAux [] = []
```

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]
```

```
powerset (x:xs) =
  [[x:xs], [x:xs], [x:xs], [x:xs], [x:xs], [x:xs], [x:xs], [x:xs]]
powerset [] = [[]]
```


Takeshi Sui
117155105

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

<pre>method f1(a:array<int>) returns (out:int) { var i:int := a.Length-1; out := 0; while (i >= 0) { out := out - a[i]; i := i-1; } }</pre>	<pre>method f2(a:array<int>) returns (out:int) { var i:int := 0; out := 0; while (i < a.Length) { out := out - a[i]; i := i+1; } }</pre>
}	}

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x\ y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x\ y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

$\text{show } a \Rightarrow a \rightarrow a \rightarrow \text{IO String}$

(d) $\backslash x\ l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) $\text{foldr } (\text{const } 1)$

ill-typed

(f) $(, "42")$

$(\text{Maybe } a, \text{String})$

(g) $(, "42")$

$(\text{Maybe } a, \text{String})$

(h) $\text{reverse} . \text{reverse}$

$\text{ord } a \Rightarrow [a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$[Int] \rightarrow [Int]$

(j) $\text{let } f\ x = x \text{ in } (f\ 'a', f\ \text{True})$

ill-typed

(k) $\text{fmap } (\text{fmap } (+1)) [\text{Nothing}]$

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) $\text{Int} \rightarrow \text{Int}$

Example answers:

$\backslash x \rightarrow x + 1$
 $(+1)$

(b) $\text{Bool} \rightarrow [\text{Bool}]$

$\backslash x \rightarrow x : [\text{true}]$

(c) $a \rightarrow \text{Maybe } b$

$\backslash x \rightarrow \text{lookup } x [\text{C1, 'one'}]$

(d) $(\text{Int} \rightarrow \text{Char} \rightarrow \text{Bool}) \rightarrow [\text{Int}] \rightarrow [\text{Char}] \rightarrow [\text{Bool}]$

$\backslash f \ x \ y \rightarrow \text{zipWith } f \ [1..x] \ ['a'..y]$

(e) $(a \rightarrow b \rightarrow c) \rightarrow \text{Maybe } a \rightarrow \text{Maybe } b \rightarrow \text{Maybe } c$

$\backslash f \ x \ y \rightarrow \text{listToMaybe } (\text{zipWith } f \ (\text{maybeToList } x) \ (\text{maybeToList } y))$

(f) $(a \rightarrow b) \rightarrow (b \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [b]$

$\backslash x \ y \ z \rightarrow \text{filter } y \ \text{map } x \ z$

(g) $\text{Maybe } a \rightarrow (a \rightarrow [b]) \rightarrow [(a,b)]$

$\backslash x \ f \rightarrow$

(h) $\text{Eq } a \Rightarrow a \rightarrow [a] \rightarrow [a]$

$\backslash x \ y \rightarrow x : \text{repeat } y$

(i) $\text{Show } a \Rightarrow [a] \rightarrow \text{IO String}$

$\backslash x \rightarrow \text{show } \text{head } x$

(j) $(a,b) \rightarrow (a \rightarrow b \rightarrow c) \rightarrow c$

$\backslash x \ f \rightarrow$

(k) $((a,b) \rightarrow c) \rightarrow c$

$\backslash f \rightarrow f ("a", 1)$

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules--write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow\rightarrow$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } }  $\rightarrow\rightarrow$ 
{ {  $0 = m * m * m$  } }
  y := 0;
{ {  $y = m * m * m$  } }
  var x := m * m;
{ {  $y = x * m$  } }
  var z := m;
{ {  $y = x * z$  } }
  while (z > 0) {
    { {  $y = x * z \ \&\& \ (z > 0) \ \&\& \ x = m * m$  } }  $\rightarrow\rightarrow$ 
    { {  $y + x = x * (z - 1) \ \&\& \ x = m * m$  } }
    z := z - 1;
    { {  $y + x = x * z \ \&\& \ x := m * m$  } }
    y := y + x;
    { {  $y = x * z \ \&\& \ x := m * m$  } }
  }
{ {  $y = x * z \ \&\& \ (z > 0) \ \&\& \ x = m * m$  } }  $\rightarrow\rightarrow$ 
{ { y = m * m * m } }
```

$$x \cdot z = x \cdot m$$

$$x \cdot m^2 = m^2 \cdot z - m^2$$

$$y + x = x * m$$

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
```

```
weave [] [] = []
```

```
weave [_] _ = []
```

```
weave _ [] = []
```

```
weave x:xs y:ys = x:y:(weave xs ys)
```

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]
```

```
powerset :: [a] -> [[a]]
```

```
powerset [] = []
```

```
powerset x:xs = zipWith x (:) powerset xs
```


CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

<pre>method f1(a:array<int>) returns (out:int) { var i : int := a.Length - 1 out := 0; while i > -1 { out := out - a[i]; } }</pre>	<pre>method f2(a:array<int>) returns (out:int) { var i : int := 0; out := 0; while i < a.Length { out := out - a[i]; } }</pre>
---	---

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x\ y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x\ y \rightarrow$ if $x == y$ then show x else show (x,y)

$\text{Eq } a, \text{Show } a \Rightarrow a \rightarrow a \rightarrow \text{String}$

(d) $\backslash x\ l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) $\text{foldr} (\text{const } 1)$

~~ill-typed~~

(f) $(, "42")$

~~ill-typed~~

— (g) $(, "42")$

$\text{String} \rightarrow (\text{String}, \text{String})$

(h) $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter} (< 42) (l ++ l)$

$[Int] \rightarrow [Int]$

— (j) $\text{let } f\ x = x \text{ in } (f\ 'a', f\ \text{True})$

~~ill-typed~~

— (k) $\text{fmap} (\text{fmap} (+1)) [\text{Nothing}]$

~~ill-typed~~

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the `appendix`; do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

Example answers:

`\x -> x + 1`

`(+1)`

(b) `Bool -> [Bool]`

`repeat True`

(c) `a -> Maybe b`

`\x -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`zipWith (\i c -> i > 0 << c == '1') [1,2,3] ['a', 'b', 'c']`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`\f (Maybe x) (Maybe y) -> Maybe (f x y)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`\f1 f2 xs -> if (f2 $1 (head xs)) then [f1 (head xs)] else [f1 (head xs)]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

`\x f -> [(x, head $ a)]`

(h) `Eq a => a -> [a] -> [a]`

`delete`

(i) `Show a => [a] -> IO String`

`\l -> show (head l)`

(j) `(a,b) -> (a -> b -> c) -> c`

`\(a,b) f -> f a b`

(k) `((a,b) -> c) -> c`

`undefined`

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
weave [] [] = []
weave _ [] = []
weave [] _ = []
weave (x:xs) (y:ys) = x:y : (weave xs ys)
```

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

```
powerset :: [a] -> [[a]]
```

```
powerset (x:xs) = helper x (x:xs) [] where
```

```
  helper :: a -> [a] -> [[a]] -> [[a]]
```

```
  helper x [] [] = [[x]]
```

```
  helper x [] [c1] [c2] = [x] : [c1]
```

```
  helper x [c1] acc = acc
```

```
  helper x (y:ys) [] = [x] : map (\y -> x:y:D) (ys:ys) : [c1]
```

```
  helper x (y:ys) [z:zs] = [x] : map (\y -> x:y:D) (ys:ys) & map (\z -> x:z:D) (z:zs) : [c1]
```


Aziel Belle

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)
```

```
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0
```

```
f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
```

```
  var i: int := a.length-1;
```

```
  out := 0;
```

```
  while (i >= 0) {
```

```
    out := out - a[i];
```

```
    i := i - 1;
```

```
  }
```

```
method f2(a:array<int>) returns (out:int) {
```

```
  var i: int := 0;
```

```
  out := 0;
```

```
  while (i < a.length) {
```

```
    out := out - a[i];
```

```
    i := i + 1;
```

```
  }
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

$a \rightarrow b \rightarrow \text{String}$

(d) $\backslash x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) foldr (const 1)

ill-typed

(f) ("42")

ill-typed

(g) $()^{\text{"42"}}$

ill-typed

(h) reverse . reverse

$[a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$[a] \rightarrow [a]$

(j) $\text{let } f \ x = x \text{ in } (f 'a', f \text{True})$

$(a \rightarrow b \rightarrow c) \rightarrow (c, c)$

(k) $\text{fmap (fmap (+1)) [Nothing]}$

ill-typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[], Nothing`, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

Example answers:

`\x -> x + 1`

`(+1)`

(b) `Bool -> [Bool]`

~~X~~ `\x -> [x]`

(c) `a -> Maybe b`

~~X~~ `\x -> Just x`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

~~X~~ `f. [] -> [false]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

~~X~~ `f. [] -> [false]`

~~X~~ `a b -> Just (f a b)`

~~X~~ `f -> [false]`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

~~X~~ `f (a:as) (b:bs) = (f a b) : x f as bs`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

~~X~~ `f1 f2 [] = []`

~~X~~ `f1 f2 (x:xs) = if f2 (f1 x) then`

(h) `Eq a => a -> [a] -> [a]`

~~X~~ `Nothing f = zip [Nothing] (f Nothing) (f1 x) : (x f1 f2 xs)`

(i) `Show a => [a] -> IO String`

~~X~~ `Just a f zip [a] (f a)`

~~X~~ `\x -> show [x]`

(j) `(a,b) -> (a -> b -> c) -> c`

~~X~~ `(a,b) f = f a b`

(k) `((a,b) -> c) -> c`

~~X~~ `=`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;  $x = 4$ 
  var z := m;  $z = 2$ 
  while (z > 0)
  {
    z := z - 1;
    y := y + x;  $y = 8$ 
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } }  $\rightarrow$ 
{ { m > 0 } }  $\&\&$   $0 + m * m + m * m = 0$ 
  y := 0;
{ { m > 0 } }  $\&\&$   $y + m * m + m * m = m$ 
  var x := m * m;
{ { m > 0 } }  $\&\&$   $y + x = m$ 
  var z := m;
{ { z > 0 } }  $\&\&$   $y + x + x = m$ 
  while (z > 0) {
    { { z > 0 } }  $\&\&$   $y + x + x = m$ 
    { { z - 1 > 0 } }  $\&\&$   $y + x + x = m$ 
      z := z - 1;
    { { z > 0 } }  $\&\&$   $y + x + x = m$ 
      y := y + x;
    { { z > 0 } }  $\&\&$   $y + x = m$ 
  }
  { { z > 0 } }  $\&\&$   $y + x = m \&\& (z > 0)$ 
  { { y = m * m * m } }  $\rightarrow$ 
```

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:
- ```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

`weave [] [] = []`  
`weave (a:as) (b:bs) = (a:b) : weave as bs`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3]]
```

`powerset [] = []`  
`powerset x = map (map) x`

`map (map) x =`



Elaine Gao

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int 0 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100
f1 = foldr (-) 0

f2 :: [Int] -> Int 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
 foldl
 var i := 0;
 out := 0;
 while (i < a.length) {
 out := out - a[i];
 i := i + 1;
 }
}

method f2(a:array<int>) returns (out:int) {
 foldl
 var i := 0;
 out := 0;
 while (i < a.length) {
 out := out - a[i];
 i := i + 1;
 }
}
```

check:

|     |   |      |
|-----|---|------|
| out | i | a[i] |
| 0   | 1 | 3    |
| 3   | 2 | 2    |
| -1  |   |      |

## Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a)  $\lambda x \rightarrow x$

Example answer:

$a \rightarrow a$

(b)  $\lambda x y \rightarrow (x, y)$

$a \rightarrow b \rightarrow (a, b)$

(c)  $\lambda x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x, y)$

$a \rightarrow a \rightarrow \text{String}$

(d)  $\lambda x l \rightarrow x : (l ++ [x])$

$a \rightarrow [a] \rightarrow [a]$

(e)  $\text{foldr } (\text{const } 1)$

$a \rightarrow [a, a, a] \rightarrow [a]$

(f)  $(\text{"42"})$

$a \rightarrow (a, \text{String})$

(g)  $(\text{"42"})$

$b \rightarrow (\text{String}, b)$

(h)  $\text{reverse, reverse}$

$[a] \rightarrow [a] \rightarrow [a] \rightarrow [a]$

(i)  $\lambda l \rightarrow \text{filter } (< 42) [1] ++ [1]$

$[Int] \rightarrow [Int]$

(j)  $\text{let } f x = x \text{ in } (f 'a', f \text{True})$

(k)  $\text{fmap } (\text{fmap } (+1)) [\text{Nothing}]$

$\text{fmap } (a \rightarrow b) \rightarrow f a \rightarrow f b$

$\lambda x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x, y)$

$a \rightarrow [a] \rightarrow [a]$

concat lists

$[1] ++ [2] = [1, 2]$

$b \rightarrow a \rightarrow b \rightarrow (a, b)$

$([a] \rightarrow [a]) \rightarrow ([a] \rightarrow [a]) \rightarrow [a] \rightarrow [a]$

$[a] \rightarrow [a] \rightarrow [a]$

$\lambda l \rightarrow \text{filter } (< 42) [1] ++ [1]$

one list [c]

you have

foldr (+)

$\text{map } (+1) [1, 2, 3]$

in out



### Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

*Example answers:*

`\x -> x + 1`  
`(+1)`

(b) `Bool -> [Bool]`  
`\x -> repeat [False]`

(c) `a -> Maybe b`  
`\x -> Just y`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`  
`zip [0..] [0..] [0..]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`  
`zip`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

(h) `Eq a => a -> [a] -> [a]`

(i) `Show a => [a] -> IO String`  
`putStrLn`

(j) `(a,b) -> (a -> b -> c) -> c`

(k) `((a,b) -> c) -> c`

### Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
 requires m > 0
 ensures y == m*m*m
{
 y := 0;
 var x := m * m;
 var z := m;
 while (z > 0)
 {
 z := z - 1;
 y := y + x;
 }
}
```

$m$   $\frac{x}{2}$   $y$   $z > 0$

$2$   $4$   $0$   $2$

$2$   $4$   $4$   $1$

$2$   $4$   $8$   $0$

$m$   $\frac{x}{3}$   $y$   $z$

$3$   $6$   $0$   $3$

$3$   $6$   $6$   $2$

$3$   $6$   $12$   $1$

$3$   $6$   $18$   $0$

$y = x \cdot m$   $z = 0$

$no z? WRONG$

$0-3 \cdot 6 = 0$

$(3-2) \cdot 6 = 6$

$y = (3-1) \cdot 6 = 12$

$y = (3-0) \cdot 6 = 18$

$y = (m-z) \cdot x$  / INVARIANT

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules - write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with  $\rightarrow$ . These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } } $\rightarrow$
{ { m > 0 } } $\wedge$ $0 == (m-m) \cdot m \cdot m$
 y := 0;
{ { m > 0 } } $\wedge$ $y == (m-m) \cdot m \cdot m$
 var x := m * m;
{ { m > 0 } } $\wedge$ $y == (m-m) \cdot x$
 var z := m;
{ { z > 0 } } $\wedge$ $y == (m-z) \cdot x$
 while (z > 0) {
 z := z - 1;
 z > 0 $\wedge$ $y == m - z \cdot x$ $\wedge$ $z > 0$
 z > 0 $\wedge$ $y + x == (m-z-1) \cdot x$
 z := z - 1;
 z > 0 $\wedge$ $y + x == (m-z) \cdot x$
 y := y + x;
 z > 0 $\wedge$ $y == (m-z) \cdot x$
 }
{ { z > 0 } } $\wedge$ $y == (m-z) \cdot x$ $\wedge$ $z > 0$
 }
{ { y = m * m * m } }
```



## Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

```
weave _ [] = []
weave [] _ = []
weave (x:xs) (y:ys) = x:y:(weave xs ys)
```

- (b) Implement a function `powerSet`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerSet [1,2,3] = [[], [1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

```
powerSet [] = []
powerSet (x:xs) = foldr ((*) 1 []) (++) powerSet xs
```

*(\*) \acc = (some way to split the values of xs and group them with x) : acc*

*[1,2,3]*

*[2,3]*

*add to the acc*



Zisheng Chen

117100433

## CMSC 433: Midterm Exam (Spring 2024)

### Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

|                                                                                                                                                                         |                                                                                                                                                                    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>method f1(a:array&lt;int&gt;) returns (out:int) {   out := 0;   var i : int = a.length - 1;   while i &gt;= 0 {     out := a[i] - out;     i := i - 1;   } }</pre> | <pre>method f2(a:array&lt;int&gt;) returns (out:int) {   out := 0;   var i : int = 0;   while i &lt; a.length {     out := out - a[i];     i := i + 1;   } }</pre> |
| }                                                                                                                                                                       | }                                                                                                                                                                  |

## Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a)  $\backslash x \rightarrow x$

*Example answer:*

$a \rightarrow a$

(b)  $\backslash x y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a, b)$

(c)  $\backslash x y \rightarrow$  if  $x == y$  then show  $x$  else show  $(x,y)$

$a \rightarrow b \rightarrow \text{String}$

(d)  $\backslash x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e)  $\text{foldr} (\text{const } 1)$

$a \rightarrow a \rightarrow [a] \rightarrow a$

(f)  $(\text{"42"})$

$a \rightarrow (a, \text{String})$

(g)  $(\text{"42"})$

$a \rightarrow (\text{String}, a)$

(h)  $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i)  $\backslash l \rightarrow \text{filter} (< 42) (l ++ l)$

$[Int] \rightarrow [Int]$

(j)  $\text{let } f \ x = x \text{ in } (f 'a', f \text{ True})$

$\text{ill-typed}$

(k)  $\text{fmap} (\text{fmap} (+1)) [\text{Nothing}]$

$[Int]$

$a \rightarrow b \rightarrow a$

$(a \rightarrow b \rightarrow b) \rightarrow b \rightarrow [a] \rightarrow b$

$\text{fmap} (\text{fmap} (+1)) [\text{Nothing}]$

$\text{maybe } \text{Int} \rightarrow f \text{ But}$

$\text{maybe } \text{Int}$

$\frac{f h}{f + g b}$

### Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a)  $\text{Int} \rightarrow \text{Int}$

*Example answers:*

$\backslash x \rightarrow x + 1$

$(+1)$

(b)  $\text{Bool} \rightarrow [\text{Bool}]$

$\backslash x \rightarrow [x \ \&\& \ \text{True}]$

(c)  $a \rightarrow \text{Maybe } b$

$\backslash x \rightarrow \text{Nothing}$

(d)  $(\text{Int} \rightarrow \text{Char} \rightarrow \text{Bool}) \rightarrow [\text{Int}] \rightarrow [\text{Char}] \rightarrow [\text{Bool}]$

$f \text{ } (g = \backslash x \rightarrow \backslash y \rightarrow (x == 0) \&\& (y == 'a')) \text{ } zs \text{ } ws = \text{zipWith } g \text{ } (0:zs) \text{ } ('a':ws)$

(e)  $(a \rightarrow b \rightarrow c) \rightarrow \text{Maybe } a \rightarrow \text{Maybe } b \rightarrow \text{Maybe } c$

$f \text{ } g \text{ } x \text{ } y = g \text{ } x \text{ } y \text{ where } g \text{ } x \text{ } y = \text{case } (x, y) \text{ of}$

$(f) \text{ } (a \rightarrow b) \rightarrow (b \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [b]$

$f \text{ } g \text{ } h \text{ } (x:xs) = \text{if } h \text{ } x \text{ then } g \text{ } x \text{ else } g \text{ } x$

(g)  $\text{Maybe } a \rightarrow (a \rightarrow [b]) \rightarrow [a, b]$

$f \text{ } x \text{ } g = [(x, z)] \text{ where } g = \text{case } x \text{ of}$

(h)  $\text{Eq } a \Rightarrow a \rightarrow [a] \rightarrow [a]$

$f \text{ } x \text{ } xs = x:xs$

(i) Show  $a \Rightarrow [a] \rightarrow \text{IO String}$

$f \text{ } (x:xs) = \text{Show } a \text{ } ++ \text{ } f \text{ } xs$

(j)  $(a, b) \rightarrow (a \rightarrow b \rightarrow c) \rightarrow c$

$\backslash (x, y) \rightarrow \backslash f \text{ } z \text{ } w = z + w \rightarrow f \text{ } x \text{ } y$

(k)  $((a, b) \rightarrow c) \rightarrow c$

$\backslash f \text{ } (x, y) = x + y \rightarrow x + y$

### Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
 requires m > 0
 ensures y == m*m*m
{
 y := 0;
 var x := m * m;
 var z := m;
 while (z > 0)
 {
 z := z - 1;
 y := y + x;
 }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules — write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with  $\rightarrow>$ . These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } } $\rightarrow>$
{ { 0 == m * m * (m - m) } && m >= 0 && m * m == m * m } }
 y := 0;
 { { y == m * m * (m - m) } && m >= 0 && m * m == m * m } }
 var x := m * m;
 { { y == x * (m - m) } && m >= 0 && x == m * m } }
 var z := m;
 { { y == x * (m - z) } && z >= 0 && x == m * m } }
 while (z > 0) {
 { { y == x * (m - z) } && z >= 0 && x == m * m } } $\rightarrow>$
 { { y + x == x * (m - (z - 1)) } && z >= 0 && x == m * m } }
 z := z - 1;
 { { y + x == x * (m - z) } && z >= 0 && x == m * m } }
 y := y + x;
 { { y == x * (m - z) } && z >= 0 && x == m * m } }
 }
 { { y == x * (m - z) } && z >= 0 && x == m * m } } $\rightarrow>$
 { { y = m * m * m } }
 }
```

|               |       |       |
|---------------|-------|-------|
| $y$           | $x$   | $z$   |
| 0             | $m^2$ | $m$   |
| $m^2$         | $m^2$ | $m-1$ |
| $2m^2$        | $m^2$ | $m-2$ |
| $3m^2$        |       | $m-3$ |
| $m \cdot m^2$ |       | 0     |

$$y = x * (m - z)$$

$$3m^2 = m^2 * (m - m + 3)$$



### Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

```
weave [] = []
weave [1] = [1]
weave (x:xs) (y:ys) = x:(y:weave xs ys)
```

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

```
powerset [] = []
powerset (x:xs) = map f powerset xs ++ powerset xs
 where f :: [Int] -> [Int]
 f ys = x:ys
```

```
p (3:[1]) = [[], [1], [3], [1,3]]
p (x:[a:b]) = [[], [a], [b], [a,b]] ++ [[x], [x,a], [x,b], [x,a,b]]
```





# CMSC 433: Midterm Exam (Spring 2024)

## Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

|                                                                                                                                                                                                   |                                                                                                                                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>method f1(a:array&lt;int&gt;) returns (out:int) {     var i := 0;     out := 0;     while i &lt; a.length - 1 {         out := out - a[i];         i := i + 1;     }     return out; }</pre> | <pre>method f2(a:array&lt;int&gt;) returns (out:int) {     var i := a.length - 1;     out := 0;     while (i &gt;= 0) {         out := out - a[i];         i := i - 1;     }     return out; }</pre> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a)  $\lambda x \rightarrow x$

*Example answer:*

$a \rightarrow a$

(b)  $\lambda x y \rightarrow (x, y)$

$(a \rightarrow b) \rightarrow (a, b)$

$\beta\text{-red}$

(c)  $\lambda x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x, y)$

$(a \rightarrow b) \rightarrow \text{Bool} \rightarrow a \mid (a, b) \rightarrow \text{String}$

(d)  $\lambda x l \rightarrow x : l ++ l ++ [x]$

$(a \rightarrow b) \rightarrow a \rightarrow [b] \rightarrow [b] \rightarrow [b, a]$

$(+ 1) : [a] \rightarrow [a] \rightarrow [a]$

(e)  $\text{foldr} (\text{const } 1)$

$(a) \rightarrow a$

(f)  $(\cdot "42")$

$(a \rightarrow \text{Char})$

(g)  $(\cdot "42")$

*ill-typed*

(h)  $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a] \rightarrow [a]$

(i)  $\lambda l \rightarrow \text{filter} (< 42) (l ++ l)$

$(a \rightarrow (a \rightarrow \text{Bool})) \rightarrow [a] \rightarrow [a]$

(j)  $\text{let } f \ x = x \text{ in } (f 'a', f \text{ True})$

$(a \rightarrow b) \rightarrow b \rightarrow (Char, (Bool))$

(k)  $\text{fmap} (\text{fmap} (+1)) [\text{Nothing}]$

$(Int \rightarrow Int \rightarrow [Maybe Int])$

$\text{foldr} : a \rightarrow (a \rightarrow b \rightarrow b) \rightarrow b \rightarrow [a] \rightarrow b$   
 $\text{const} :: a \rightarrow b \rightarrow a$

$[a] \rightarrow [a]$

$a \rightarrow a \rightarrow a$

### Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the `appendix`, `do` syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

*Example answers:*

`\x -> x + 1`  
`(+1)`

(b) `Bool -> [Bool]`

`\x -> repeat 1 x`

(c) `a -> Maybe b`

`foo x = Nothing | Just True`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`zipWith 2 'x' True`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`foo x y z = Nothing | Just x`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`func f x=y = zipWith (x y)`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

`foo a = [True]`

(h) `Eq a => a -> [a] -> [a]`

`delete 3 [1,2,3,4]`

(i) `Show a => [a] -> IO String`

`[x] -> x ++ "string"`

(j) `(a,b) -> (a -> b -> c) -> c`

`\x.y -> zipWith True`

(k) `((a,b) -> c) -> c`

`\(x,y) -> True == True`

### Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
 requires m > 0
 ensures y == m*m*m
{
 y := 0;
 var x := m * m;
 var z := m;
 while (z > 0)
 {
 z := z - 1;
 y := y + x;
 }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with  $\rightarrow>$ . These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } } $\rightarrow>$
{ { 0 = m * m * m } }
 y := 0;
{ { y = m * m * m } }
 var x := m * m;
{ { y = x * m } }
 var z := m;
{ { y = x * m } }
 while (z > 0) {
 { { y = x * m } }
 { { y + x = x * m } }
 z := z - 1;
 { { y + x = x * m } }
 y := y + x;
 { { y = x * m } }
 }
{ { y = x * m } }
{ { y = m * m * m } } $\rightarrow>$
```

$$\begin{array}{l}
 y \\
 0 \\
 0 + m^2 \\
 m^2 + m^2 \\
 3m^2 \\
 m \cdot m^2
 \end{array}
 \quad
 \begin{array}{l}
 x \\
 m^2 \\
 z \\
 m \\
 m-1 \\
 m-2 \\
 m-3 \\
 0
 \end{array}
 \quad
 \begin{array}{l}
 m^2 \\
 m^2 \\
 m^2 \\
 m^2 \\
 m^2 \\
 m^2 \\
 m^2
 \end{array}$$

## Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave :: [Int] -> [Int] -> [Int]`

`weave f [x:xs] (y:ys) = weave (f x y) (xs ++ ys)`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`powerset :: [Int] -> [[Int]]`

`powerset f (x:xs) (y:ys) = powerset (x, y) (xs ++ ys)`





# Tests Review

## CMSC 433: Midterm Exam (Spring 2024)

### Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

|                                                                                                                                                                                     |                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>method f1(a:array&lt;int&gt;) returns (out:int) {   var i := a.length - 1   var out := 0   while (i &gt; 0) {     out := a[i] - a[i-1]     i := i - 1   }   return out }</pre> | <pre>method f2(a:array&lt;int&gt;) returns (out:int) {   var i := 0   var out := 0   while (i &lt; a.length - 1) {     out := a[i] - a[i+1]     i := i + 1   }   return out }</pre> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`  
*Example answer:*  
`a -> a`

(a) `\x -> x`  
*Example answer:*  
`a -> a`

(b) `\x y -> (x,y)`

`Int -> a -> (Int, a)`

(c) `\x y -> if x == y then show x else show (x,y)`

`Int -> String`

(d) `\x l -> x : l ++ l ++ [x]`

`Int -> [Int]`

(e) `foldr (const 1)`

`(a -> b -> b) -> b -> [a] -> b`

(f) `(^42)`

`Int -> Int`

(g) `(^42)`

`Int -> Int`

(h) `reverse . reverse`

`[a] -> [a] -> [a] -> [a]`

(i) `\l -> filter (< 42) (l ++ l)`

`[a] -> (a -> Bool) -> [a] -> [a]`

(j) `let f x = x in (f 'a', f True)`

(k) `fmap (fmap (+1)) [Nothing]`

`(a -> b) -> [a] -> [Maybe b]`



### Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

*Example answers:*

`\x -> x + 1`

`(+1)`

$x \rightarrow x + 5$

(b) `Bool -> [Bool]`

$x \rightarrow \text{if } x \text{ then } [0] \text{ else } [0]!x$

(c) `a -> Maybe b`

$x \rightarrow \text{Just } b$

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

$\text{zipWith } 5 \text{ 'a' True}$

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

$\text{zipWith } \text{Just } u \text{ Just } y \text{ Just } c$

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

(h) `Eq a => a -> [a] -> [a]`

(i) `Show a => [a] -> IO String`

(j) `(a,b) -> (a -> b -> c) -> c`

(k) `((a,b) -> c) -> c`



### Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave [] [] = []`

`weave (x:xs) (y:ys) = x:y:weave xs ys`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[], [1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`powerset [] = [[]]`

`powerset [1] = [[]]`

`powerset`



Parav Rathore

117761214

## CMSC 433: Midterm Exam (Spring 2024)

### Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

|                                                                                                                                                                  |                                                                                                                                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>method f1(a:array&lt;int&gt;) returns (out:int) {   var i:int := a.length-1;   out := 0;   while i &gt; 0 {     out := out - a[i];     i = i-1;   } }</pre> | <pre>method f2(a:array&lt;int&gt;) returns (out:int) {   var i:int := 0;   out := 0;   while i &lt; a.length {     out := out - a[i];     i = i+1;   } }</pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|

### Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or “ill-typed” if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a)  $\backslash x \rightarrow x$

*Example answer:*

$a \rightarrow a$

(b)  $\backslash x\ y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c)  $\backslash x\ y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(d)  $\backslash x\ l \rightarrow x : l ++ l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e)  $\text{foldr } (\text{const } 1)$

$(a \rightarrow b \rightarrow a) \rightarrow b \rightarrow a \rightarrow b$

(f)  $(\text{"42"})$

$a \rightarrow \text{String} \rightarrow (a, \text{String})$

(g)  $(.) \text{"42"}$

$a \rightarrow \text{String} \rightarrow (a, \text{String})$

(h)  $\text{reverse} \cdot \text{reverse}$

$([b] \rightarrow [c]) \rightarrow ([a] \rightarrow [b]) \rightarrow a \rightarrow c$

(i)  $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$[a] \rightarrow (a \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [a]$

(j)  $\text{let } f\ x = x \text{ in } (f\ 'a', f\ \text{True})$

$(a \rightarrow b) \rightarrow b$

(k)  $\text{fmap } (\text{fmap } (+1))\ [\text{Nothing}]$

$a \rightarrow (a \rightarrow b) \rightarrow a \rightarrow \text{Maybe } a$



### Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

*Example answers:*

`\x -> x + 1`

`(+1)`

(b) `Bool -> [Bool]`

`\a -> if a then [ ] else [ ]`

(c) `a -> Maybe b`

`\x -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`let f x y = f (x+1) y`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`let f x y = Just (f Just x Just y)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`let f g l = if g (f (f g+1)) then [f (f g+1)] else [f (f g+1)]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

`let x f = (f (Just x))`

(h) `Eq a => a -> [a] -> [a]`

`delete x y`

(i) `Show a => [a] -> IO String`

(j) `(a,b) -> (a -> b -> c) -> c`

`let f g = g f x y`

(k) `((a,b) -> c) -> c`

### Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
 requires m > 0
 ensures y == m*m*m
{
 y := 0;
 var x := m * m;
 var z := m;
 while (z > 0)
 {
 z := z - 1;
 y := y + x;
 }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules---write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with  $\rightarrow\triangleright$ . These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } } $\rightarrow\triangleright$
{ { m > 0 } } 0 + (m * m) = (m - 1) * m * m
{
 y := 0;
 { { y + (m * m) = (m * m) * m } }
 var x := m * m;
 { { y + x = x * m - 1 * m } } & m > 0
 var z := m;
 { { y + x = x * z - 1 * m } } & m > 0
 while (z > 0) { { y + x = x * z - 1 * m } } & z > 0
 { { y + x = x * z - 1 * m } } & z - 1 >= 0
 { { y + x = x * z - 1 * m } } & z >= 0
 { { y := y + x; } }
 { { y = x * m * m } } & z >= 0
}
{ { ! (z > 0) } } & y = x * m * m
{ { y = m * m * m } } $\rightarrow\triangleright$
```



### Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
weave (x:xs) (y:ys) = (x:y) ++ weave xs ys
```

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]
powerset :: [a] -> [[a]]
powerset [] = []
powerset (x:xs) = (x:xs) ++ powerset xs
```



Donah Cantelmi

UID 117187403

## CMSC 433: Midterm Exam (Spring 2024)

### Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

|                                                                                                                                         |                                                                                                                              |
|-----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <pre>method f1(a:array&lt;int&gt;) returns (out:int) {   out := 0   for i in range(0, a.length)     out = out - a[a.length-1-i] }</pre> | <pre>method f2(a:array&lt;int&gt;) returns (out:int) {   out := 0   for i in range(0, a.length)     out = out + a[i] }</pre> |
|-----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|

const :: a -> b -> a

### Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or “ill-typed” if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a)  $\backslash x \rightarrow x$

*Example answer:*

$a \rightarrow a$

(b)  $\backslash x\ y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c)  $\backslash x\ y \rightarrow$  if  $x == y$  then show  $x$  else show  $(x,y)$

(d)  $\backslash x\ l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e)  $\text{foldr} (\text{const } 1)$

(f)  $(\cdot, "42")$

(g)  $(\cdot, "42")$

(h)  $\text{reverse} \cdot \text{reverse}$

(i)  $\backslash l \rightarrow \text{filter} (< 42) (l ++ l)$

(j)  $\text{let } f\ x = x \text{ in } (f\ 'a', f\ \text{True})$

(k)  $\text{fmap} (\text{fmap} (+1)) [\text{Nothing}]$

### Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

*Example answers:*

`\x -> x + 1`  
`(+1)`

(b) `Bool -> [Bool]`

`\b -> [b]`

(c) `a -> Maybe b`

`\x -> if x == x then Just x else Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`\f ints chers -> map f (mapf ints)`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`\f a b -> if isJust a && isJust b then f a b else Nothing`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`\f1 f2 list -> map f1 list`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

(h) `Eq a => a -> [a] -> [a]`

(i) `Show a => [a] -> IO String`

(j) `(a,b) -> (a -> b -> c) -> c`

(k) `((a,b) -> c) -> c`

### Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
 requires m > 0
 ensures y == m*m*m
{
 y := 0;
 var x := m * m;
 var z := m;
 while (z > 0)
 {
 z := z - 1;
 y := y + x;
 }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence so that if you work backwards from the end of the program you should only rely on all the usual rules of arithmetic.

```
{ { m > 0 } } ->>
{ {
 {
 y := 0;
 var x := m * m;
 var z := m;
 { {
 while (Z > 0) {
 { { z <= m & z >= 0
 { { y = x * (m - z)
 z := z - 1;
 } {
 y := y + x;
 }
 }
 }
 { { y == x * m
 { { y = m * m * m } }
 }
 }
 } } ->>
```



Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave [] [] = []`  
`weave (x:xs) (y:ys) = [x,y] ++ (weave xs ys)`

- (b) Implement a function `powerSet`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerSet [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`  
`powerSet [] = [[]]`  
`powerSet (x:xs) = [x] ++ [x:xs] ++ powerSet xs`





Olivia Beck  
117132233

## CMSC 433: Midterm Exam (Spring 2024)

### Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

$[1, 2, 3] = 0 - 3 - 2 - 1$

$[1, 2, 3] = 0 - 1 - 2 - 3$

Write two Daffy programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

| method <code>f1(a:array&lt;int&gt;)</code> returns (out:int) {                              | method <code>f2(a:array&lt;int&gt;)</code> returns (out:int) {                                                              |
|---------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <pre>out := 0 var i := 0 while i &lt; a.length {   out := out - a[i];   i := i + 1; }</pre> | <pre>out := 0 var i := a.length - 1 if i == 0 {   return out; } while i &gt; 0 {   out := out - a[i];   i := i - 1; }</pre> |
| }                                                                                           | }                                                                                                                           |

## Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a)  $\backslash x \rightarrow x$

*Example answer:*

$a \rightarrow a$

(b)  $\backslash x y \rightarrow (x, y)$

$a \rightarrow b \rightarrow (a, b)$

\* (c)  $\backslash x y \rightarrow$  if  $x == y$  then show  $x$  else show  $(x, y)$

$a \rightarrow a \rightarrow \text{String}$

$(a) ++ [a] ++ [a]$   
 $[a] ++ [a] = [a]$

(d)  $\backslash x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

\* (e)  $\text{foldr} (\text{const } 1) \text{acc} [x] \rightarrow [y]$

$(b \rightarrow \text{Int}) \rightarrow [b] \rightarrow (b \rightarrow \text{Int})$

(f)  $(\text{"42"}) \rightarrow (,) \rightarrow a \rightarrow b \rightarrow (a, b)$

$a \rightarrow (a, \text{String})$

(g)  $(.) \text{"42"}$

$b \rightarrow (\text{String}, b)$

(h)  $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i)  $\backslash l \rightarrow \text{filter} (< 42) (l ++ l)$

$[\text{Int}] \rightarrow [\text{Int}]$

(j)  $\text{let } f \ x = x \ \text{in } (f \ 'a', f \ \text{True})$

$(\text{Char}, \text{Bool})$

\* (k)  $\text{fmap} (\text{fmap} (+1)) [\text{Nothing}] \rightarrow \text{Maybe} (\text{fmap} (+1)) \Rightarrow$

$[\text{Maybe Int}]$

### Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the `appendix`, `do` syntax, `list` comprehensions, or any valid Haskell.

(a) `Int -> Int`

*Example answers:*

`\x -> x + 1`  
`(+1)`

(b) `Bool -> [Bool]`

`\b -> [b] :: True]`

~~(c) `a -> Maybe b`~~

~~`\a -> Nothing`~~

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

~~`\f xs cs -> if (head xs == '1' && head cs == 'c') then [(f 1 'c') && True]`~~  
~~`else [False]`~~

~~`zipWith (return Maybe)`~~

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`\f g as -> if g (f (head as)) == True then [f (head as)] else`

`[f (last as)]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

`let f (Just x) g = [(x, head (g x))]`

(h) `Eq a => a -> [a] -> [a]`

`\a as -> if head as == a then [a] else as`

(i) `Show a => [a] -> IO String`

(j) `(a,b) -> (a -> b -> c) -> c`

`\(a,b) f -> f a b`

(k) `((a,b) -> c) -> c`



## Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave [] [] = []`  
`weave (1:rs) (r:rs) = 1:r:(weave rs rs)`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[], [1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`powerset [] = [[]]`  
`powerset [x] = [[]x]`  
`powerset (x:xs) = [x:(powerset xs)] : (powerset xs)`





Molly Wilson  
118001463

## CMSC 433: Midterm Exam (Spring 2024)

### Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldl (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

|                                                                                                                                                                      |                                                                                                                                                                 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>method f1(a:array&lt;int&gt;) returns (out:int) {   var i := a.length - 1;   out := 0;   while (i &gt;= 0) {     out := out - a[i];     i := i - 1;   } }</pre> | <pre>method f2(a:array&lt;int&gt;) returns (out:int) {   var i := 0;   out := 0;   while (i &lt; a.length) {     out := out - a[i];     i := i + 1;   } }</pre> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a)  $\backslash x \rightarrow x$

*Example answer:*

$a \rightarrow a$

(b)  $\backslash x\ y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c)  $\backslash x\ y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

$a \rightarrow a \rightarrow \text{String}$

(d)  $\backslash x\ l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e)  $\text{foldr } (\text{const } 1)$

$(a \rightarrow b \rightarrow b) \rightarrow b$

(f)  $(\cdot) "42"$

$a \rightarrow \text{String} \rightarrow (a, \text{String})$

(g)  $(\cdot) "42"$

$a \rightarrow \text{String} \rightarrow (a, \text{String})$

(h)  $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a] \rightarrow ([a] \rightarrow [a]) \rightarrow [a] \rightarrow [a]$

(i)  $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$[a] \rightarrow [a] \rightarrow [a]$

(j)  $\text{let } f\ x = x \text{ in } (f\ 'a', f\ \text{True})$

$a \rightarrow a$

(k)  $\text{fmap } (\text{fmap } (+1)) [\text{Nothing}]$

$(a \rightarrow b) \rightarrow [\text{Maybe } a] \rightarrow [\text{Maybe } b]$



### Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

*Example answers:*

`\x -> x + 1`  
`(+1)`

(b) `Bool -> [Bool]`

`\x -> [x && False]`

(c) `a -> Maybe b`

`Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`zipWith (\x,y -> x+1==4 || y=='a') [1,2,3] ['a','b','c']`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`fun f a b = case (a, b) of (Maybe x, maybe y) -> Nothing (c)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`-1 -> Nothing`

`filter (\x -> True) map (\y -> 3) ["abc"]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

`fun a f = case a of map x -> foldr (ie a -> (x, e) acc) [] (f x)`

(h) `Eq a => a -> [a] -> [a]`

`delete x 1`

`delete 1`

(i) `Show a => [a] -> IO String`

`print "hi"`

(j) `(a,b) -> (a -> b -> c) -> c`

`fun (a,b) f = f a b`

(k) `((a,b) -> c) -> c`

`f repeat, repeat,`

### Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
 requires m > 0
 ensures y == m*m*m
{
 y := 0;
 var x := m * m;
 var z := m;
 while (z > 0)
 {
 z := z - 1;
 y := y + x;
 }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with  $\rightarrow\gg$ . These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } } $\rightarrow\gg$
{ { 0 = 0 } } $\&\&$ 0 = m * m * (m - m)
 y := 0;
 { { y = m * m * (m - m) } }
 var x := m * m;
 { { y = x * (m - m) } }
 var z := m;
 { { y = x * (m - z) } }
 while (z > 0) {
 { { z > 0 } } $\&\&$ y = x * (m - z)
 { { z > 0 } } $\&\&$ y + x = x * (m - (z - 1))
 z := z - 1;
 { { z > 0 } } $\&\&$ y + x = x * (m - z)
 y := y + x;
 { { z > 0 } } $\&\&$ y = x * (m - z) + 1
 }
 { { !(z > 0) } } $\&\&$ y = x * (m - z)
 }
 { { y = m * m * m } } $\rightarrow\gg$
```

### Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave [] [] = []
weave (1:12) (1:12) = [1,1,2,2]
weave (1:12) [] = [1:12]
weave [] (1:12) = [1:12]
weave (1:12) (1:12) = [1,1,2,2,3,3,4,4,5,5,6,6]
```

- (b) Implement a function `powerSet`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerSet [1,2,3] = [[], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3], []]
```

```
powerSet [] = []
powerSet [1] = [[], [1]]
powerSet [1,2] = [[], [1], [2], [1,2]]
powerSet [1,2,3] = [[], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3]]
powerSet [1,2,3,4] = [[], [1], [2], [3], [4], [1,2], [1,3], [1,4], [2,3], [2,4], [3,4], [1,2,3], [1,2,4], [1,3,4], [2,3,4], [1,2,3,4]]
```



Matthew Nashel  
117283501

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
 var z := a.Length-1;
 out := 0;
 while z >= 0 {
 out := out - a[z];
 z := z - 1;
 }
}
```

```
method f2(a:array<int>) returns (out:int) {
 var z := 0;
 out := 0;
 while z < a.Length {
 out := out - a[z];
 z := z + 1;
 }
}
```

## Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`

*Example answer:*

`a -> a`

(b) `\x y -> (x,y)`

`a -> b -> (a,b)`

(c) `\x y -> if x == y then show x else show (x,y)`

`a -> a -> String`

(d) `\x l -> x : l ++ l ++ [x]`

`a -> [a] -> [a]`

(e) `foldr (const 1)`

`ill-typed`

(f) `(, "42")`

`ill-typed`

(g) `(,) "42"`

`ill-typed`

(h) `reverse . reverse`

`[a] -> [a]`

(i) `\l -> filter (< 42) (l ++ l)`

`[Int] -> [Int]`

(j) `let f x = x in (f 'a', f True)`

`ill-typed`

(k) `fmap (fmap (+1)) [Nothing]`

`ill-typed`



### Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[], Nothing`, or undefined) unless there is no other option. You can use any function from the `appendix`, `do` syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

*Example answers:*

`\x -> x + 1`

`(+1)`

(b) `Bool -> [Bool]`

(c) `a -> Maybe b`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

(h) `Eq a => a -> [a] -> [a]`

*delete*

(i) `Show a => [a] -> IO String`

(j) `(a,b) -> (a -> b -> c) -> c`

(k) `((a,b) -> c) -> c`

*f -> f (x,y)*

### Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
 requires m > 0
 ensures y == m*m*m
{
 y := 0;
 var x := m * m;
 var z := m;
 while (z > 0)
 {
 z := z - 1;
 y := y + x;
 }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post-conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with  $\Rightarrow$ . These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } } $\Rightarrow$
{ { m > 0 } } $\wedge$ $z = 0$
 y := 0;
 { { m > 0 } } $\wedge$ $m \cdot m = z \cdot m \cdot m$
 var x := m * m;
 { { x == m * m } } $\wedge$ $m > 0$ $\wedge$ $m == z \cdot m$
 var z := m;
 { { m > 0 } } $\wedge$ $z == m$ $\wedge$ $x == m \cdot m$
 while (z > 0)
 {
 { { z > 0 } } $\wedge$ $x == m \cdot m$ $\wedge$ $z == z \cdot m$ } $\Rightarrow$
 { { z - 1 + 1 == z } }
 { { z + 1 == z } } $\wedge$ $y + x == m \cdot m$
 y := y + x;
 { { y == (z + 1) * (z + 1) } } $\wedge$ $z + 1$
 }
 { { y == (z + 1) * (z + 1) } } $\wedge$ $z - 1$ } $\Rightarrow$
 }
 { { y == m * m * m } }
```



## Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave :: [a] -> [a] -> [a]`  
`weave [] [] = []`  
`weave (x:xs) (z:zs) = [x] ++ [z] ++`  
`weave xs zs`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`powerset :: [a] -> [[a]]`

`powerset [] = []`

`powerset (x:xs) = [x] ++ head xs`  
`++ tail xs ++`  
`powerset xs`



Iskan Kar

## CMSC 433: Midterm Exam (Spring 2024)

### Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
 out := 0;
 i := a.length - 1;
 while (i >= 0) {
 out := out + a[i];
 i := i - 1;
 }
}

method f2(a:array<int>) returns (out:int) {
 out := 0;
 i := 0;
 while (i < a.length) {
 out := out + a[i];
 i := i + 1;
 }
}

}
```

### Question 2 (20 points)

For each of the Haskell expressions below, write their (most **general**) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are **provided** in the appendix at the end.

(a)  $\backslash x \rightarrow x$

*Example answer:*

$a \rightarrow a$

(b)  $\backslash x\ y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c)  $\backslash x\ y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

$a \rightarrow a \rightarrow \text{String}$

(d)  $\backslash x\ l \rightarrow x : l ++ l ++ [x]$

*ill-typed*

(e)  $\text{foldr} (\text{const } 1)$

*ill-typed*

(f)  $(\text{"42"})$

$b \rightarrow (\text{String}, b)$

(g)  $(\text{"42"})$

$b \rightarrow (\text{Int}, b)$

(h)  $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i)  $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$[\text{Int}] \rightarrow [\text{Int}]$

(j)  $\text{let } f\ x = x \text{ in } (f\ 'a', f\ \text{True})$

*ill-typed*

(k)  $\text{fmap } (\text{fmap } (+1)) [\text{Nothing}]$

$[\text{Maybe Int}]$

### Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) Int -> Int

*Example answers:*

\x -> x + 1

(+1)

(b) Bool -> [Bool]

\x -> if x then [x] else []

(c) a -> Maybe b

\y -> Nothing

(d) (Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]  
 \f a b -> case (a,b) of (x,y) -> let v = y; "hi" in [if f x y then [x y] else []]

(e) (a -> b -> c) -> Maybe a -> Maybe b -> Maybe c  
 \f a b -> case (a,b) of (Just x, Just y) -> Just (f x y)  
 \_, \_ -> Nothing

(f) (a -> b) -> (b -> Bool) -> [a] -> [b]  
 \f g lst -> map f (filter (f.g) lst)

(g) Maybe a -> (a -> [b]) -> [(a,b)]  
 \f -> case of (Just a) of (x:xs) -> [(a,x)]  
 Just a -> (case f a of [c] -> [c])  
 Nothing -> []

(h) Eq a => a -> [a] -> [a]  
 \a lst -> if a == a then a:lst else lst

(i) Show a => [a] -> IO String  
 \lst -> do map (putStrLn . show) lst

(j) (a,b) -> (a -> b -> c) -> c  
 \((a,b)) f -> f a b

(k) ((a,b) -> c) -> c  
 \f = undefined

### Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
 requires m > 0
 ensures y == m*m*m
{
 y := 0;
 var x := m * m;
 var z := m;
 while (z > 0)
 {
 z := z - 1;
 y := y + x;
 }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules - write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with  $\rightarrow>$ . These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } } $\rightarrow>$
{ { 0 == 0 xx m > 0 } }
 y := 0;
{ { y == 0 xx m > 0 } }
 var x := m * m;
{ { y + x == x xx m > 0 } }
 var z := m;
{ { y + x == (m - (z - 1)) xx xx z > 0 } }
 while (z > 0) {
 { { y == (m - z) xx xx z > 0 } } $\rightarrow>$
 { { y + x == (m - (z - 1)) xx xx z > 0 } }
 z := z - 1;
 { { y + x == (m - z) xx xx z > 0 } }
 y := y + x;
 { { y == (m - z) xx xx z > 0 } }
 }
 { { y == (m - z) xx xx z == 0 } } $\rightarrow>$
 { { y = m * m * m } }
```



### Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

```
weave [] [] = []
weave [x] [] = [x]
weave [] [y] = [y]
weave (x:xs) (y:ys) = x:y:(weave xs ys)
```

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[], [1,2,3], [1,2], [1,3], [2,3], [2], [3], []]
for every element, y in a list where the element is not the last
powerset (x:xs) = [(powerset xs)] + [(map (++) [x]) powerset xs]
powerset [] = []
```

```
powerset [1] = [[]] + [[1]] = [[], [1]]
powerset [2,1] =
```





Tyler Grove

YTD

117263909

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)
```

```
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0
```

```
f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Daffy programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
 requires a.length > 0
```

```
 var i: Int := a.length - 1;
 out := 0;
 while i >= 0
 invariant -1 <= i <= a.length - 1
 {
 out := out - a[i];
 i := i - 1;
 }
}
```

```
method f2(a:array<int>) returns (out:int) {
 requires a.length > 0
```

```
 var i: Int := 0;
 out := 0;
 while i < a.length
 invariant 0 <= i <= a.length
 {
 out := out + a[i];
 i := i + 1;
 }
}
```

## Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a)  $\backslash x \rightarrow x$

*Example answer:*

$a \rightarrow a$

(b)  $\backslash x y \rightarrow (x,y)$

*tuple*

$a \rightarrow b \rightarrow (a,b)$

(c)  $\backslash x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

$a \rightarrow a \rightarrow \text{IO } ()$

(d)  $\backslash x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e)  $\text{foldr } (\text{const } 1)$

$b \rightarrow \text{Int} \rightarrow a$

(f)  $(\text{"42"})$

*ill-typed*

(g)  $() \text{"42"}$

$a \rightarrow (\text{String}, a)$

(h)  $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i)  $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$[a] \rightarrow [a]$

(j)  $\text{let } f \ x = x \text{ in } (f \ 'a', f \ \text{True})$

$(\text{Char}, \text{Bool})$

(k)  $\text{fmap } (\text{fmap } (+1)) [\text{Nothing}]$

*ill-typed*

$(\backslash (a \rightarrow b) \rightarrow [a] \rightarrow [b]) \rightarrow [a] \rightarrow [b]$

### Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a)  $\text{Int} \rightarrow \text{Int}$

*Example answers:*

$\backslash x \rightarrow x + 1$   
 $(+1)$

(b)  $\text{Bool} \rightarrow [\text{Bool}]$

$\backslash x \rightarrow [x]$

(c)  $a \rightarrow \text{Maybe } b$

*undefined*

(d)  $(\text{Int} \rightarrow \text{Char} \rightarrow \text{Bool}) \rightarrow [\text{Int}] \rightarrow [\text{Char}] \rightarrow [\text{Bool}]$

$\backslash f \text{ I C} \rightarrow \text{zipWith } f \text{ I C}$

(e)  $(a \rightarrow b \rightarrow c) \rightarrow \text{Maybe } a \rightarrow \text{Maybe } b \rightarrow \text{Maybe } c$

$\backslash f \text{ ab} \rightarrow (q \text{ of } \text{Nothing} \rightarrow \text{Nothing}) \text{ Just } q \rightarrow (q \text{ of } \text{Nothing} \rightarrow \text{Nothing})$

(f)  $(a \rightarrow b) \rightarrow (b \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [b]$

$\backslash f \text{ bq} \rightarrow \text{map } f \text{ q}$  Just b  $\rightarrow \text{Just } (f \text{ q b})$

(g)  $\text{Maybe } a \rightarrow (a \rightarrow [b]) \rightarrow [(a, b)]$

$\backslash q f \rightarrow (q \text{ of } \text{Nothing} \rightarrow []) \text{ Just } q \rightarrow \text{foldr } (\backslash x b \rightarrow (q, x), b)$

(h)  $\text{Eq } a \Rightarrow a \rightarrow [a] \rightarrow [a]$

$\backslash q l \rightarrow q : l$

(i)  $\text{Show } a \Rightarrow [a] \rightarrow \text{IO String}$

$\backslash q \rightarrow \text{do } \text{print } q \text{ } | \text{ "hello"}$

(j)  $(a, b) \rightarrow (a \rightarrow b \rightarrow c) \rightarrow c$

$\backslash (q, b) f \rightarrow f \text{ q b}$

(k)  $((a, b) \rightarrow c) \rightarrow c$

*undefined*

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
 requires m > 0
 ensures y == m*m*m
{
 y := 0;
 var x := m * m;
 var z := m;
 while (z > 0)
 {
 z := z - 1;
 y := y + x;
 }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post-conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with  $\rightarrow$ . These implication steps can (silently) rely on all the usual rules of arithmetic.

```

{ { m > 0 } } -->
{ { 0 = (m - m) * m * m } }
{ { y := 0; } }
{ { y = (m - m) * m * m } }
{ { var x := m * m; } }
{ { y = (m - m) * x } }
{ { var z := m; } }
{ { y = (m - z) * x } }
{ { while (z > 0) { } } }
{ { (y + x) = (m - (z - 1)) * x } }
{ { y = (m - z) * x } }
{ { z := z - 1; } }
{ { (y + x) = (m - z) * x } }
{ { y := y + x; } }
{ { y = (m - z) * x } }
{ { y = (m - z) * x } }
{ { y = (m - z) * x } }

```

Why (none)  
Is first?

$$w * w * (\overline{Z} - w) = Y$$

### Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave :: [a] -> [a] -> [a]`

`Weave a b = foldr (\(q1,q2) b -> [q1,q2] ++ b) [] (zip a b)`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`Powerset :: [a] -> [[a]]`

`Powerset a = foldr (\x b -> (foldr (\y c -> (x:y) ++ c) [] b) ++ b) [] a`

~~`foldr`~~

appends old list with result of  
second foldr





CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Datix programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

|                                                                                                                                                                                                        |                                                                                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>r method f1(a:array&lt;int&gt;) returns (out:int) {     var i:int := a.length-1;     out := 0;     while (i &gt;= 0) {         out := out - a[i];         i := i-1;     }     return out; }</pre> | <pre>method f2(a:array&lt;int&gt;) returns (out:int) {     var i:int := 0;     out := 0;     while (i &lt; a.length){         out := out - a[i];         i := i+1;     }     return out; }</pre> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a)  $\backslash x \rightarrow x$

*Example answer:*

$a \rightarrow a$

(b)  $\backslash x y \rightarrow (x, y)$

$a \rightarrow b \rightarrow (a, b)$

(c)  $\backslash x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x, y)$

$a \rightarrow a \rightarrow \text{Unit}$

(d)  $\backslash x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e)  $\text{foldr } (\text{const } 1) \text{ } b \text{ } l$

$a \rightarrow \text{Int} \rightarrow \text{Int}$

(f)  $(\text{"42"})$

$(a, \text{String})$

(g)  $(\text{"42"})$

$(a, b) \rightarrow \text{String}$

(h)  $\text{reverse} . \text{reverse}$

$(f \cdot g)$

(i)  $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$[\text{Int}] \rightarrow [\text{Int}]$

(j)  $\text{let } f \ x = x \text{ in } (f 'a', f \text{ True})$

$(a \rightarrow a) \rightarrow a \rightarrow (a, b)$

(k)  $\text{fmap } (\text{fmap } (+1)) [\text{Nothing}]$

$[\text{Int}] \rightarrow \text{Maybe Int}$



### Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as []). Nothing, or undefined unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

*Example answers:*

`\x -> x + 1`

`(+1)`

(b) `Bool -> [Bool]`

`\x x = x && True : []`

(c) `a -> Maybe b`

undefined

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`f (x:xs) (y:ys) -> (f x y) : []`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`\f x y -> f (Just a) (Just b)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

undefined

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

undefined

(h) `Eq a => a -> [a] -> [a]`

`\x x xs = x : [xs]`

(i) `Show a => [a] -> IO String`

`putStr [1,2,3]`

(j) `(a,b) -> (a -> b -> c) -> c`

`fun tup f = let (a,b) = tup in (f a b)`

(k) `((a,b) -> c) -> c`

undefined

### Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int) $z > 0$
 requires $m \geq 0$.
 ensures $y == m * m * m$
{
 y := 0;
 var x := $m * m$; x^2
 var z := m; $y = 0$
 while (z > 0)
 {
 $x + m = y$ $x \neq m^2$
 $z := z - 1$;
 $y := y + x$;
 }
}
```

$m \quad x \quad y \quad z$

|   |   |    |   |
|---|---|----|---|
| 3 | 9 | 0  | 3 |
| 2 | 9 | 18 | 1 |
| 1 | 9 | 27 | 0 |

$x = y / m^2$   
 $27 / 9 = 3$   
 $m = y / m^2$   
 $x = y * z$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence so that if you work backwards from the end of the program you should only rely on all the usual rules of arithmetic.

```
{ { m > 0 } } ->>
{ { 0 = m * m * m } }
{ { y := 0;
 var x := m * m;
 var z := m;
 while (z > 0) {
 { { y = m * m * x } }
 { { y + x = m * x } }
 z := z - 1;
 { { y + x = m * x } }
 y := y + x;
 { { y = m * x } }
 }
 { { y = m * x } }
 { { y = m * m * x } } ->>
```

### Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
xs ys
```

You can assume that the lists have the same length.

```
weave [] = []
weave [x] = [x]
weave (x:xs) (y:ys) = [x] ++ [y] ++ weave xs ys
```

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[], [1], [2], [1,3], [1,2], [3], [2,3], [1,2,3]]
```

```
powerset [] = []
```



# CMSC 433: Midterm Exam (Spring 2024)

## Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

|                                                                                                                                                                    |                                                                                                                                                                 |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>method f1(a:array&lt;int&gt;) returns (out:int) {   var i:int := a.length-1;   out := 0;   while i &gt;= 0 {     out := a[i] - out;     i := i-1;   } }</pre> | <pre>method f2(a:array&lt;int&gt;) returns (out:int) {   var i:int := 0;   out := 0;   while i &lt; a.length {     out := out - a[i];     i := i+1;   } }</pre> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are **provided** in the appendix at the end.

(a)  $\lambda x \rightarrow x$

*Example answer:*

$a \rightarrow a$

(b)  $\lambda x y \rightarrow (x.y)$

$a \rightarrow b \rightarrow (a,b)$

(c)  $\lambda x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x.y)$

$\text{Eq } a, \text{ show } a \Rightarrow a \rightarrow a \rightarrow \text{String}$

(d)  $\lambda x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e)  $\text{foldr } (\text{const } 1)$

$\text{Foldable } t \Rightarrow \text{Int} \rightarrow t \rightarrow \text{Int} \rightarrow \text{Int}$

(f)  $(^"42")$

*ill-typed*

(g)  $(.) ^"42"$

$b \rightarrow (\text{String}, b)$

(h)  $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i)  $\lambda l \rightarrow \text{filter } (< 42) (l ++ l)$

$[a] \rightarrow [a]$

(j)  $\text{let } f \ x = x \text{ in } (f 'a', f \text{True})$

$(\text{Char}, \text{Bool})$

(k)  $\text{fmap } (\text{fmap } (+1)) [\text{Nothing}]$

$[ \text{Maybe Int} ]$

$(a \rightarrow b) \rightarrow f a \rightarrow f b$   
 $\quad \quad \quad \text{List}$   
 $\quad \quad \quad \text{Maybe}$   
 $(a \rightarrow b) \rightarrow f a \rightarrow f b$   
 $\text{Int} \rightarrow \text{Int} \quad \text{Maybe Int} \quad \text{Maybe Int}$



### Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a)  $\text{Int} \rightarrow \text{Int}$

*Example answers:*

$\backslash x \rightarrow x + 1$

$(+1)$

(b)  $\text{Bool} \rightarrow [\text{Bool}]$

*repeat True*

(c)  $a \rightarrow \text{Maybe } b$

*$\backslash x \rightarrow \text{Nothing}$*

(d)  $(\text{Int} \rightarrow \text{Char} \rightarrow \text{Bool}) \rightarrow [\text{Int}] \rightarrow [\text{Char}] \rightarrow [\text{Bool}]$

*$\backslash f \text{ is } cs \rightarrow ((f!1!a)!!\text{True}) : \text{zipWith } f \text{ is } cs)$*

(e)  $(a \rightarrow b \rightarrow c) \rightarrow \text{Maybe } a \rightarrow \text{Maybe } b \rightarrow \text{Maybe } c$

*$\text{fmap } \backslash f \ x \ y \rightarrow \text{if isJust } \text{LiftM2 } f \ x \ y \text{ then liftM2}(f \ x \ y) \text{ else Nothing}$*

(f)  $(a \rightarrow b) \rightarrow (b \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [b]$

*$\backslash f \ g \ as \ bs \mapsto \text{filter } g \ (\text{map } f \ as \ bs)$*

(g)  $\text{Maybe } a \rightarrow (a \rightarrow [b]) \rightarrow [(a,b)]$

*$\backslash m \ f \rightarrow \text{case } m \text{ of Just } x \rightarrow \text{zip } (\text{repeat } x) \ (f \ x) \text{ Nothing } \rightarrow []$*

(h)  $\text{Eq } a \Rightarrow a \rightarrow [a] \rightarrow [a]$

*$\backslash x \rightarrow \text{filter } (== x)$*

(i) Show  $a \Rightarrow [a] \rightarrow \text{IO String}$

*map (putStrLn show)*

(j)  $(a,b) \rightarrow (a \rightarrow b \rightarrow c) \rightarrow c$

*$\backslash (x,y) \ f \rightarrow f \ x \ y$*

(k)  $((a,b) \rightarrow c) \rightarrow c$

*$\backslash f \rightarrow f \ (\text{undefined}, \text{undefined})$*

### Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
 requires m > 0
 ensures y == m*m*m
{
 y := 0;
 var x := m * m;
 var z := m;
 while (z > 0)
 {
 z := z - 1;
 y := y + x;
 }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules— write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence so that if you work backwards from the end of the program you should only rely on all the usual rules of arithmetic.

```
{ { m > 0 } } -->
{ { m > 0 } } || (m - m) * (m * m) == 0 && m * m == m * m
 y := 0;
 { { m > 0 } } && (m - m) * (m * m) == y && m * m == m * m
 var x := m * m;
 { { m > 0 } } && (m - m) * x == y && x == m * m
 var z := m;
 { { z > 0 } } && (m - z) * x == y && x == m * m
 while (z > 0) {
 { { z > 0 } } && (m - z) * x == y && x == m * m && z > 0
 { { z - 1 > 0 } } && (m - (z - 1)) * x == y + x && x == m * m
 z := z - 1;
 { { z > 0 } } && (m - z) * x == y + x && x == m * m
 y := y + x;
 { { z > 0 } } && (m - z) * x == y && x == m * m
 }
 { { z > 0 } } && (m - z) * x == y && x == m * m && z > 0
 }
 } -->
 }
 } -->
 }
}
```

$x$   
 $m^2$   
 $m$

$y$   
 $0$

$z$   
 $m$

$m^2$   
 $m^3$   
 $0$



## Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

*weave :: [Int] -> [Int] -> [Int]*  
`weave (x:xs) (y:ys) = x:(y:(weave xs ys))`  
`weave [] [] = []`  
`weave (x:xs) [] = x:xs`  
`weave [] (y:ys) = y:ys`

*Should never happen but for completeness*

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[], [1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

*Scratch work*

*build it up*

`powerset (x:xs) = let r = (powerset xs) in r ++ (zipWith (:) (repeat x) r)`  
`powerset [] = [[]]`

*Scratch work*

`powerset [] = [[]]`  
`powerset [3] = [] + [3] ← from 3:[] = [3], []`  
`[2,3] = [] + [2,3], [2] ← from 2:[] = [2,3], [2], []`  
`from 2: [3]`  
`[3], []`



## Prelude Functions and Datatypes

### Booleans

```
data Bool = False | True

(&&) :: Bool -> Bool -> Bool
(||) :: Bool -> Bool -> Bool
not :: Bool -> Bool
```

### Lists

```
data [] a = [] | a : [a]

(++) :: [a] -> [a] -> [a]
head, last :: [a] -> a
tail, init :: [a] -> [a]

null :: Foldable t => t a -> Bool
length :: Foldable t => t a -> Int
map :: (a -> b) -> [a] -> [b]
reverse :: [a] -> [a]
interperse :: a -> [a] -> [a]
transpose :: [[a]] -> [[a]]
foldl :: Foldable t => (b -> a -> b) -> b -> t a -> b
foldr :: Foldable t => (a -> b -> b) -> b -> t a -> b
concat :: Foldable t => t [a] -> [a]
concatMap :: (a -> [b]) -> [a] -> [b]
concatMapM :: (a -> [b] -> m b) -> [a] -> m [b]
and, or :: Foldable t => t Bool -> Bool
any, all :: Foldable t => (a -> Bool) -> t a -> Bool
sum, product :: (Foldable t, Num a) => t a -> a
maximum, minimum :: (Foldable t, Ord a) => t a -> a
repeat :: a -> [a]
replicate :: Int -> a -> [a]
take, drop :: Int -> [a] -> [a]
splitAt :: Int -> [a] -> ([a], [a])
takeWhile, dropWhile :: (a -> Bool) -> [a] -> [a]
group :: Eq a => [a] -> [[a]]
inits, tails :: [a] -> [[a]]
elem, notElem :: (Foldable t, Eq a) => a -> t a -> Bool
lookup :: Eq a => a -> [(a, b)] -> Maybe b
find :: Foldable t => (a -> Bool) -> t a -> Maybe a
filter :: (a -> Bool) -> [a] -> [a]
zip :: [a] -> [b] -> [(a, b)]
zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
nub :: Eq a => [a] -> [a]
delete :: Eq a => a -> [a] -> [a]
sort :: Ord a => [a] -> [a]
sortBy :: Ord a => (a -> b) -> [a] -> [a]
```

### Maybe

```
data Maybe a = Nothing | Just a

maybe :: b -> (a -> b) -> Maybe a -> b
isJust, isNothing :: Maybe a -> Bool
listToMaybe :: [a] -> Maybe a
maybeToList :: Maybe a -> [a]
catMaybes :: [Maybe a] -> [a]
mapMaybe :: (a -> Maybe b) -> [a] -> [b]
```

## Typeclass Definitions

```
class Semigroup a where
 (<>) :: a -> a -> a

class Semigroup m => Monoid m where
 mempty :: m

class Show a where
 show :: a -> String

class Eq a where
 (==) :: a -> a -> Bool
 (/=) :: a -> a -> Bool

class Eq a => Ord a where
 compare :: a -> a -> Ordering
 (<), (<=), (>=), (>) :: a -> a -> Bool
 max, min :: a -> a -> a

class Functor f where
 fmap :: (a -> b) -> f a -> f b

class Functor f => Applicative f where
 pure :: a -> f a
 (<*>) :: f (a -> b) -> f a -> f b

class Applicative m => Monad m where
 return :: a -> m a
 (>>=) :: m a -> (a -> m b) -> m b

class Foldable t where
 foldMap :: Monoid m => (a -> m) -> t a -> m

class Arbitrary a where
 arbitrary :: Gen a
 shrink :: a -> [a]

class Num a where
 (+), (-), (*) :: a -> a -> a
 negate :: a -> a
 abs :: a -> a
 signum :: a -> a
 fromInteger :: Integer -> a
```

## Functors, Applicatives, and Monads

```
(<$>) :: Functor f => (a -> b) -> f a -> f b
(<$) :: Functor f => a -> f b -> f a
(>$) :: Functor f => f a -> b -> f b

liftA2 :: Applicative f => (a -> b -> c) -> f a -> f b -> f c
(*) :: Applicative f => f a -> f b -> f b
(<*) :: Applicative f => f a -> f b -> f a
when :: Applicative f => Bool -> f () -> f ()

(>>) :: Monad m => m a -> m b -> m b
mapM :: Monad m => (a -> m b) -> [a] -> m [b]
filterM :: Monad m => (a -> m Bool) -> [a] -> m [a]
sequence :: Monad m => [m a] -> m [a]
join :: Monad m => m (m a) -> m a
zipWithM :: Monad m => (a -> b -> m c) -> [a] -> [b] -> m [c]
foldM :: (Foldable t, Monad m) => (b -> a -> m b) -> b -> t a -> m b
replicateM :: Applicative m => Int -> m a -> m [a]

liftM :: Monad m => (a1 -> r) -> m a1 -> m r
liftM2 :: Monad m => (a1 -> a2 -> r) -> m a1 -> m a2 -> m r
...
```

## 1 Hoare Rules

$$\frac{\begin{array}{l} \{P\} s1 \{Q\} \\ \{Q\} s2 \{R\} \end{array}}{\{P\} s1; s2 \{R\}} \quad (\text{hoare\_seq})$$
$$\frac{}{\{Q \ [x := a]\} x := a \{Q\}} \quad (\text{hoare\_asgn})$$
$$\frac{\begin{array}{l} \{P'\} c \{Q'\} \\ P \Rightarrow P' \quad Q' \Rightarrow Q \end{array}}{\{P\} c \{Q\}} \quad (\text{hoare\_consequence})$$
$$\frac{\begin{array}{l} \{P \ \&\& \ b\} s1 \{Q\} \\ \{P \ \&\& \ !b\} s2 \{Q\} \end{array}}{\{P\} \text{if } b \{ s1 \} \text{ else } \{ s2 \} \{Q\}} \quad (\text{hoare\_if})$$
$$\frac{\{P \ \&\& \ b\} s \{P\}}{\{P\} \text{while } b \{ s \} \{P \ \&\& \ !b\}} \quad (\text{hoare\_while})$$



## CMSC 433: Midterm Exam (Spring 2024)

## Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

|                                                                                                                                                                          |                                                                                                                                                                       |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>method f1(a:array&lt;int&gt;) returns (out:int) {   var index := a.length-1;   while (index &gt;= 0) {     out := out - a[index];     index := index-1;   } }</pre> | <pre>method f2(a:array&lt;int&gt;) returns (out:int) {   var index := 0;   while (index &lt; a.length) {     out := out - a[index];     index := index+1;   } }</pre> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a)  $\backslash x \rightarrow x$

*Example answer:*

$a \rightarrow a$

(b)  $\backslash x y \rightarrow (x,y)$

$a \rightarrow [b] \rightarrow [(b,a)]$

(c)  $\backslash x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

ill-typed

(d)  $\backslash x l \rightarrow x : l ++ l ++ [x]$

$[a] \rightarrow [a] \rightarrow [a]$

(e)  $\text{foldr } (\text{const } 1)$

$\text{Int} \rightarrow [\text{Int}] \rightarrow \text{Int}$

(f)  $(^* 42)$

$([a], [a])$

$> [4 2]$

(g)  $(^* 42)$

$a \rightarrow [a] \rightarrow [a]$

(h)  $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i)  $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$[1, 2, 3, 4, 5]$

$[a] \rightarrow (a \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [a]$

(j)  $\text{let } f \ x = x \text{ in } (f 'a', f \text{ True})$

$(a \rightarrow a) \rightarrow a \rightarrow a$

(k)  $\text{fmap } (\text{fmap } (+1)) [\text{Nothing}]$

ill-typed



### Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

*Example answers:*

`\x -> x + 1`

`(+1)`

(b) `Bool -> [Bool]`

`\x -> [x]`

(c) `a -> Maybe b`

`\x -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`  
`bunc b a b = map (\(x,y) -> b x y) (zip a b)`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`bunc b a b = case a of` *Maybe Just a -> case b of* *Maybe Just b -> b a b*  
*case Nothing -> Nothing* *case Nothing -> Nothing*

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`  
`(billr (<20)).(map (+1))`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`  
*undefined*

(h) `Eq a => a -> [a] -> [a]`  
`delete 'e' ['a','e','i','o']`

(i) `Show a => [a] -> IO String`  
`fmap (\a -> show a)`

(j) `(a,b) -> (a -> b -> c) -> c`  
`\a b b -> b a b`

(k) `((a,b) -> c) -> c`  
`(\a,b -> b a b)`

### Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
```

```
 requires m > 0
```

```
 ensures y == m*m*m
```

```
{
```

```
 y := 0;
```

```
 var x := m * m;
```

```
 var z := m;
```

```
 while (z > 0)
```

```
 {
```

```
 z := z - 1;
```

```
 y := y + x;
```

```
 }
```

```
}
```

m = 3

| z   | y                              | x              |          |
|-----|--------------------------------|----------------|----------|
| 3   | 0                              | m <sup>2</sup> | 2 9 9 9  |
| m-1 | m <sup>2</sup>                 | m <sup>2</sup> | 2 9 9 9  |
| m-2 | m <sup>2</sup> +m <sup>2</sup> | m <sup>2</sup> | 1 18 9 9 |
|     |                                |                | 0 27 9 9 |

$Inv \rightarrow y = m^3 \wedge z$

$y := x * (m - z)$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with  $\rightarrow$ . These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } } ->>
{ { 0 == m * m * (m - m) }
 y := 0;
 { { y == m * m * (m - m) }
 var x := m * m;
 { { y == x * (m - m) }
 var z := m;
 { { y == x * (m - z) }
 while (z > 0) {
 { { y == x * (m - z) } } ->>
 { { y + x == x * (m - z + 1) }
 z := z - 1;
 { { y == y + x; }
 { { y == x * (m - z) }
 y := x * (m - z);
 }
 }
 { { y == x * (m - z) } } ->>
 { { y == m * m * m } }
 }
 }
 }
 }
}
```

## Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave _ [] = []
weave [] _ = []
weave (x:xs) (y:ys) = x:y:(weave xs ys)
```

- (b) Implement a function `powerSet`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerSet [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]
```

```
powerSet [] = []
powerSet list = powerSetAux list []
where
 powerSetAux [] [] b = b
 powerSetAux [a] [a] b = [a]:b
 powerSetAux (x:xs) b = (x:xs):powerSet xs b
```



Olade Kounou

1178744126

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
 var i = a.length - 1;
 out = 0;
 while i >= 0 {
 out = a[i] - out;
 i = i - 1;
 }
 return out
}
```

```
method f2(a:array<int>) returns (out:int) {
 var i = 0;
 out = 0;
 while i < a.length {
 out = out - a[i];
 i = i + 1;
 }
 return out
}
```

}

}

### Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a)  $\backslash x \rightarrow x$

*Example answer:*

$a \rightarrow a$

(b)  $\backslash x y \rightarrow (x,y)$

$(,)$

(c)  $\backslash x y \rightarrow$  if  $x == y$  then show  $x$  else show  $(x,y)$

$a \rightarrow a \rightarrow \text{String}$

(d)  $\backslash x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) `foldr (const J)`

`J` -  $t_1 \rightarrow t_2$

(f) `("42")`

$[Int]$

(g) `(,) "42"`

$(a)$

(h) `reverse . reverse`

$[c] \rightarrow [a]$

(i)  $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$[Int] \rightarrow Int \rightarrow Bool \rightarrow (Int) \rightarrow [Int]$

(j) `let f x = x in (f 'a', f True)`

$Bool \rightarrow Bool$

(k) `fmap (fmap (+1)) [Nothing]`

$[a] \rightarrow [Maybe b]$



### Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

*Example answers:*

`\x -> x + 1`

`(+1)`

(b) `Bool -> [Bool]`

`True : []`

(c) `a -> Maybe b`

`x = Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`zipWith (|, "a", True)`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`listToMaybe (a, b, c)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`filter (>.)`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

`maybeToList (,)`

(h) `Eq a => a -> [a] -> [a]`

`\x -> x:[ ]`

(i) `Show a => [a] -> IO String`

`putStr`

(j) `(a,b) -> (a -> b -> c) -> c`

(k) `((a,b) -> c) -> c`

`(*) " X "`

### Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
 requires m > 0
 ensures y == m*m*m
{
 y := 0;
 var x := m * m;
 var z := m;
 while (z > 0)
 {
 z := z - 1;
 y := y + x;
 }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post-conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with  $\rightarrow\gg$ . These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } } $\rightarrow\gg$
{ { 0 = (m * m) * (m - m) } }
 y := 0;
 { { y = (m * m) * (m - m) } }
 var x := m * m;
 { { y = x * (m - m) } }
 var z := m;
 { { y = x * (m - z) } }
 while (z > 0) {
 { { y = x * (m - z) } }
 { { y + x = x * (m - (z - 1)) } }
 { { z := z - 1; } }
 { { y + x = x * (m - z) } }
 y := y + x;
 { { y = x * (m - z) } }
 }
 { { y = m * m * m } } $\rightarrow\gg$
```

$$y = 0$$

$$y = x \cdot m$$

$$y = x \cdot z$$

$$x \cdot (m - z)$$



## Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave [] [] = []
```

```
weave (x:xs) (y:ys) = x:y:weave xs ys
```

```
1,2 : 2,5 : 3,6 : []
```

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]
```

```
powerset [] = [[]]
```

```
powerset
```

```
powerset (x:xs) = x:powerset xs
```

po

```
1,2,3,4
```

```
1,2,3
```

```
[1,2,3,4]
```

```
1,2,3,4
```



CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

|                                                                                                                                                                     |                                                                                                                                                                |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>method f1(a:array&lt;int&gt;) returns (out:int) {   var i := a.Length - 1;   out = 0;   while (i &gt;= 0) {     out := a[i] - out;     i := i - 1;   } }</pre> | <pre>method f2(a:array&lt;int&gt;) returns (out:int) {   var i := 0;   out = 0;   while (i &lt; a.Length) {     out := out - a[i];     i := i + 1;   } }</pre> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|

}

}

## Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a)  $\backslash x \rightarrow x$

*Example answer:*

$a \rightarrow a$

(b)  $\backslash x y \rightarrow (x, y)$

$a \rightarrow b \rightarrow (a, b)$

(c)  $\backslash x y \rightarrow$  if  $x == y$  then show  $x$  else show  $(x, y)$

$a \rightarrow a \rightarrow \text{String}$

(d)  $\backslash x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e)  $\text{foldr} (\text{const } 1)$

$\text{::} \text{Int} \rightarrow \text{Int}$

(f)  $(, "42")$

$a \rightarrow (a, \text{String})$

(g)  $(, "42")$

$a \rightarrow (\text{String}, a)$

(h)  $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i)  $\backslash l \rightarrow \text{filter} (< 42) (l ++ l)$

$[\text{Int}] \rightarrow [\text{Int}]$

(j)  $\text{let } f x = x \text{ in } (f 'a', f \text{True})$

$(\text{String}, \text{Bool})$

(k)  $\text{fmap} (\text{fmap} (+1)) [\text{Nothing}]$

$[\text{Maybe Int}]$

$a \rightarrow b \rightarrow \text{Int}$

$(a \rightarrow b) \rightarrow b \rightarrow [a] \rightarrow b$

$(b \rightarrow \text{Int})$

$(a \rightarrow b) \rightarrow a$

$f b$

$(a \rightarrow b) \rightarrow a$

$\text{Maybe Int}$

## Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

- (a) `Int -> Int`  
*Example answers:*  
`\x -> x + 1`  
`(+1)`
- (b) `Bool -> [Bool]`  
`\b -> b : [True]`
- (c) `a -> Maybe b`  
`\a -> Nothing`
- (d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`  
`\f !i !c -> (zipWithn f (i++[c]) (1c++[c])) ++ [True]`
- (e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`  
`\f ma mb -> if f ma mb -- not only maybe exists, so use m.a, m.b or just (a,b) instead of Nothing`
- (f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`  
`\f f !l !la -> filter f !l + (map f la)`
- (g) `Maybe a -> (a -> [b]) -> [(a,b)]`  
`\ma f -> case ma of`  
`Nothing -> []`  
`Just a -> zip [a] (fa)`
- (h) `Eq a => a -> [a] -> [a]`  
`\a la -> filter (==a) la`
- (i) `Show a => [a] -> IO String`  
`\la -> use la of`  
`CT -> return "forgot IO, oops!"`  
`(a:as) -> return (show a)`
- (j) `(a,b) -> (a -> b -> c) -> c`  
`\(a,b) f -> f a b`
- (k) `((a,b) -> c) -> c`  
`undefined`

### Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
 requires m > 0
 ensures y == m*m*m
{
 y := 0;
 var x := m * m;
 var z := m;
 while (z > 0)
 {
 z := z - 1;
 y := y + x;
 }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules – write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with  $\rightarrow>$ . These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } } $\rightarrow>$
{ { m·m=m·m && m>0 && m>0 && 0=(m-m)·m·m } }
 y := 0;
{ { m·m=m·m && m>0 && m>0 && y=(m-m)·m·m } }
 var x := m * m;
{ { x=m·m && m>0 && m>0 && y=(m-m)·x } }
 var z := m;
{ { x=m·m && m>0 && z>0 && y=(m-z)·x } }
 while (z > 0) {
 { { x=m·m && m>0 && z>0 && y=(m-z)·x && (z>0) } } $\rightarrow>$
 { { x=m·m && m>0 && z-1>0 && y+x=(m-(z-1))·x } }
 z := z - 1;
 { { x=m·m && m>0 && z>0 && y+x=(m-z)·x } }
 y := y + x;
 { { x=m·m && m>0 && z>0 && y=(m-z)·x } }
 }
 { { x=m·m && m>0 && z>0 && y=(m-z)·x && !(z>0) } } $\rightarrow>$
 { { y = m * m * m } }
```

$(m-z) \cdot x = y \wedge !(z > 0)$   
 needed extras to prove post  
 $z > 0 \quad x = m \cdot m$   
 $m > 0$  not strictly necessary but nice to keep



Svetlana Senenova

### Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave :: [a] -> [a] -> [a]`

`weave [] [] = []`

`weave (x:xs) (y:ys) = [x,y] ++ (weave xs ys)`

`weave _ _ = []` -- But should never occur

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`powerset [] = [[]]`

`powerset (x:xs) =`

`let cur = powerset xs in`

`cur ++ (map (x:) cur)`

`powerset [] = [[]]`

`[a] = [[]] + [a[]] = [[] [a]]`

`[a,b] = [[],[a]] + [a[], a[b]] = [[],[a], [a], [a,b]]`

yes ✓





Jonas Besson

UId: 118850870

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

|                                                                                                                                                                                                     |                                                                                                                                                                                                |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>method f1(a:array&lt;int&gt;) returns (out:int) {     out := 0;     var L := a.Length - 1;     while (L &gt;= 0) {         out := out - a[L];         L := L - 1;     }     return out }</pre> | <pre>method f2(a:array&lt;int&gt;) returns (out:int) {     out := 0;     var L := 0;     while (L &lt; a.Length) {         out := out - a[L];         L := L + 1;     }     return out }</pre> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or “ill-typed” if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

$$(a) \quad x - \lambda x$$

Example answer:

$$a - \frac{1}{a}$$

(b)  $\setminus_{x,y} - > (x,y)$

$$a \rightarrow b \rightarrow (a, b)$$

(c)  $x \cdot y \rightarrow$  if  $x == y$  then show  $x$  else show  $(x,y)$

$$a \Rightarrow a \rightarrow \text{String}$$

(d)  $x \mid x \leq 1$   $x : 1 \leq x$   $x \mid x \leq 1$   $x : 1 \leq x$

$$a \Rightarrow [a] \Rightarrow [a]$$

(e) `foldr (const 1)`

il-Tred

 $(f)(\cdot) \left( \begin{matrix} \cdot \\ \cdot \\ \cdot \\ \cdot \\ \cdot \end{matrix}, 42^{\cdot\cdot} \right)$ 

iii-Typed

(9) (.) "42"

$$a \rightarrow (a, \text{string})$$

(h) reverse . reverse

$$[\alpha] \rightarrow [\alpha]$$

(i)  $1 \rightarrow \text{filter}(<42)(1++1)$

$$[\alpha] \leq [\alpha]$$

(j) let  $f\ x = x$  in  $(f\ 'a', f\ True)$

(Choi, Bui)

$$(k) \text{ fmap}(\text{fmap}(+1))[\text{Nothing}]$$

25

(reverse, reverse)

reverse)

nothing, +1

### Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

*Example answers:*

`\x -> x + 1`

`(+1)`

(b) `Bool -> [Bool]`

`\x -> [x || True]`

(c) `a -> Maybe b`

`\a ->`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`\x, y, z ->`

`zipWith`

`[1,2,3] ['a','b','c']`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`\x y z -> fmap x z`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

(h) `Eq a => a -> [a] -> [a]`

`\x y -> delete x y`

(i) `Show a => [a] -> IO String`

`let f = show`

`a`

(j) `(a,b) -> (a -> b -> c) -> c`

(k) `((a,b) -> c) -> c`

### Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
 requires m > 0
 ensures y == m*m*m
{
 y := 0;
 var x := m * m;
 var z := m;
 while (z > 0)
 {
 z := z - 1;
 y := y + x;
 }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence so that if you work backwards from the end of the program you should only rely on all the usual rules of arithmetic.

```
{ { m > 0 } } -->
{ { 0 == m * m } }
 y := 0;
 { { y == (m-m) * m * m } }
 var x := m * m;
 { { y == (m-m) * x } }
 var z := m;
 { { y == (m-z) * x } }
 while (z > 0) {
 { { y == (m-z) * x } }
 { { y + x == (m-z-1) * x } } -->
 z := z - 1;
 { { y + x == (m-z) * x } }
 y := y + x;
 { { y == (m-z) * x } }
 }
 { { y == (m-z) * x } }
 { { y == m * m * m } } -->
```

$\alpha m = 2$

|   |   |   |   |
|---|---|---|---|
| m | y | z | x |
| 2 | 0 | 2 | 4 |
| 1 | 4 | 1 | 4 |
| 0 | 8 | 0 | 4 |

m=3

|   |    |   |   |
|---|----|---|---|
| m | y  | z | x |
| 3 | 0  | 3 | 9 |
| 2 | 9  | 2 | 9 |
| 1 | 18 | 1 | 9 |
| 0 | 27 | 0 | 9 |

$$x^3 = x^2 \cdot x$$

$$(m-z) \cdot x = y$$

### Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

`[1] [2]`

You can assume that the lists have the same length.

`weave [] [] = []`

`weave (x:xs) (y:ys) = x:y:(weave xs ys)`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`powerset [] = []`

`powerset a = a : powerset (tails a)`





Liam O'Donnell  
UTD: 118343402

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)
```

```
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0
```

```
f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two *Dafny* programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

|                                                                                                                                                                                       |                                                                                                                                                                                    |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>method f1(a:array&lt;int&gt;) returns (out:int) {   <del>var</del> out := 0;   var i:int := a.length-1;   while (i &gt;= 0) {     out := out - a[i];     i := i - 1;   } }</pre> | <pre>method f2(a:array&lt;int&gt;) returns (out:int) {   <del>var</del> out := 0;   var i:int := 0;   while (i &lt; a.length) {     out := out - a[i];     i := i + 1;   } }</pre> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a)  $\backslash x \rightarrow x$

*Example answer:*

$a \rightarrow a$

(b)  $\backslash x\ y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c)  $\backslash x\ y \rightarrow$  if  $x == y$  then show  $x$  else show  $(x,y)$

$a \rightarrow b \rightarrow ()$

(d)  $\backslash x\ l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) `foldr (const 1)`

$a \rightarrow [a] \rightarrow a$

(f) `( "42" )`

$String \rightarrow String$

(g) `( "42" )`

$String \rightarrow String$

(h) `reverse . reverse`

(i)  $\backslash l \rightarrow$  filter  $(< 42) (l ++ l)$

$[Int] \rightarrow [Int]$

(j) `let f x = x in (f 'a', f True)`

(k) `fmap (fmap (+1)) [Nothing]`



Liam O'Donnell

### Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a)  $\text{Int} \rightarrow \text{Int}$

*Example answers:*

$\backslash x \rightarrow x + 1$

$(+1)$

(b)  $\text{Bool} \rightarrow [\text{Bool}]$

$\backslash val \rightarrow [val]$

(c)  $a \rightarrow \text{Maybe } b$

$\backslash a \rightarrow \text{Just } a$

(d)  $(\text{Int} \rightarrow \text{Char} \rightarrow \text{Bool}) \rightarrow [\text{Int}] \rightarrow [\text{Char}] \rightarrow [\text{Bool}]$

$\text{foo } f \ (x :: xs) \ (y :: ys) = (f \ x \ y) : (\text{foo } f \ xs \ ys)$

(e)  $(a \rightarrow b \rightarrow c) \rightarrow \text{Maybe } a \rightarrow \text{Maybe } b \rightarrow \text{Maybe } c$

$\backslash f \ \backslash \text{Just } x \rightarrow \text{Just } y \rightarrow \text{Just } (f \ x \ y)$

(f)  $(a \rightarrow b) \rightarrow (b \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [b]$

$\text{foo } f1 \ f2 \ (x :: xs) = f1 \ f2 \ (f1 \ x) \text{ then } (f1 \ x) : \text{foo } f1 \ f2 \ xs$

(g)  $\text{Maybe } a \rightarrow (a \rightarrow [b]) \rightarrow [(a, b)]$

$\text{foo } (\text{Just } x) \ f = \text{helper } x \ (f \ x) \text{ where } \text{helper } y \ (x :: xs) = (y, x) : \text{helper } xs$

$\text{else } \text{foo } f1 \ f2 \ xs$

(i)  $\text{Show } a \Rightarrow [a] \rightarrow \text{IO String}$

$\backslash val \rightarrow \text{Just } \rightarrow$

(j)  $(a, b) \rightarrow (a \rightarrow b \rightarrow c) \rightarrow c$

$\backslash (x, y), f \rightarrow (f \ x \ y)$

(k)  $((a, b) \rightarrow c) \rightarrow c$

$\backslash f \rightarrow f \ a$

### Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
 requires m > 0
 ensures y == m*m*m
{
 y := 0;
 var x := m * m;
 var z := m;
 while (z > 0)
 {
 z := z - 1;
 y := y + x;
 }
}
```

$$y = (m - z) \times$$

$$\text{Invariant: } z > 0 \ \&\& \ y = (m - z) \times x$$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and, pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with  $\rightarrow>$ . These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } } $\rightarrow>$
{ { m > 0 } } $\&\& \ 0 == (z - z) * (m * m)$
{ { m > 0 } }
{ { m > 0 } } $\&\& \ y == (z * z) * (m * m)$
 var x := m * m;
 { { m > 0 } } $\&\& \ y == (z - z) * x$
 var z := m;
 { { z > 0 } } $\&\& \ y == (m - z) * x$
 while (z > 0) {
 { { z > 0 } } $\&\& \ y == (m - z) * x \ \&\& \ z > 0 \rightarrow>$
 { { z - 1 > 0 } } $\&\& \ y + x == m - (z - 1) * x$
 { { z := z - 1; } }
 { { z > 0 } } $\&\& \ y + x == z * (m - z) * x$
 y := y + x;
 { { z > 0 } } $\&\& \ y == (m - z) * x$
 }
 { { z > 0 } } $\&\& \ y == (m - z) * x \ \&\& \ (z > 0) \rightarrow>$
 { { y = m * m * m } }
```

$$0 = z = 0 \checkmark \text{ and } m \times = 0$$

Liam O'Donnell

### Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave :: [a] -> [a] -> [a]`

`weave [] [] = []`

`weave lst [] = lst`

`weave [] lst = lst`

`weave (x:xs) (y:ys) = [x] ++ [y] ++ (weave xs ys)`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`powerset :: [a] -> [[a]]`

`powerset lst = case lst of`  
`[] -> [[]]`

`(x:xs) ->`



Joel Gnanapri-Davis

## CMSC 433: Midterm Exam (Spring 2024)

### Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
 var i: int := 0;
 out := 0;
 while i < a.length {
 out := out - a[i];
 i := i + 1;
 }
}

method f2(a:array<int>) returns (out:int) {
 var i: int := 0;
 out := 0;
 while i < a.length {
 out := out - a[i];
 i := i + 1;
 }
}
```

### Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a)  $\lambda x \rightarrow x$

*Example answer:*

$a \rightarrow a$

(b)  $\lambda x y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c)  $\lambda x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

$a \rightarrow b \rightarrow \text{String}$

(d)  $\lambda x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) `foldr (const 1)`

ill-typed  $\text{const} :: a \rightarrow b \rightarrow a$

(f) `("42")`

$a \rightarrow (a, \text{String})$

(g) `(,)` `"42"`

$a \rightarrow (a, \text{String})$

(h) `reverse . reverse`

$[a] \rightarrow [c]$

(i)  $\lambda l \rightarrow \text{filter } (< 42) (l ++ l)$

$[\text{Int}] \rightarrow [\text{Int}]$

(j) `let f x = x in (f 'a', f True)`

$a \rightarrow b \rightarrow (a, b)$

(k) `fmap (fmap (+1)) [Nothing]`

ill-typed



### Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[], Nothing`, or `undefined`) unless there is no other option. You can use any function from the `appendix`, `do` syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

*Example answers:*

`\x -> x + 1`  
`(+1)`

(b) `Bool -> [Bool]`

`\x -> if x then [x]`

(c) `a -> Maybe b`

`undefined`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`\f !! !c -> if head (f ! !c) then (zipWith f ! !c`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`\f (Maybe x) (Maybe y) -> Maybe (f x y)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`\f1 f2 ! -> if head (f2 x) then x where x = map f1 !`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

`\(Maybe a) f -> map`

(h) `Eq a => a -> [a] -> [a]`

`delete`

(i) `Show a => [a] -> IO String`

`\x -> putString $ show (head x)`

(j) `(a,b) -> (a -> b -> c) -> c`

`\(x,b)`

(k) `((a,b) -> c) -> c`

`\f`

### Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
 requires m > 0
 ensures y == m*m*m
{
 y := 0;
 var x := m * m;
 var z := m;
 while (z > 0)
 {
 z := z - 1;
 y := y + x;
 }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules — write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence so that if you work backwards from the end of the program you should only rely on all the usual rules of arithmetic.

```
{ { m > 0 } } ->>
{ { 0 <= m } } && 0 = m * 1 * m } }
{ { y := 0; } }
{ { 0 <= m } } y = m * m * m } }
{ { var x := m * m; } } y = x * m } }
{ { var z := m; } } y = x * z } }
{ { while (z > 0) { } } }
{ { y = x * z } } z > 0 } }
{ { y + x = (x * z - 1) } } z := z - 1; } }
{ { y + x = x * z } } y := y + x; } }
{ { y = x * z } } } }
{ { z = m } } y = x * z && ! (z > 0) } }
{ { y = m * m * m } } ->>
```



### Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave [] [] = []`

`weave (x:xs) (y:ys) = [x,y] ++ (weave xs ys)`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3]]`

`powerset [] = []`

`powerset (x:xs) =`



Anthony Chitic

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0 X acc

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

|                                                                                                                                                                |                                                                                                                                                               |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>method f1(a:array&lt;int&gt;) returns (out:int) {   var i := a.Length;   out := 0;   while i &gt;= 0 {     out := a[i] - out;     i := i - 1;   } }</pre> | <pre>method f2(a:array&lt;int&gt;) returns (out:int) {   var i := 0;   out := 0;   while i &lt; a.Length {     out := a[i] - out;     i := i + 1;   } }</pre> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|

}

}

### Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a)  $\backslash x \rightarrow x$

*Example answer:*

$a \rightarrow a$

(b)  $\backslash x\ y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c)  $\backslash x\ y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

$a \rightarrow a \rightarrow ()$

(d)  $\backslash x\ l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e)  $\text{foldr } (\text{const } 1)$

$a \rightarrow t \rightarrow a \rightarrow \text{Int}$

(f)  $(\cdot 42^n)$

$a \rightarrow (a, \text{String})$

(g)  $(\cdot) \cdot 42^n$

$a \rightarrow (\text{String}, a)$

(h)  $\text{reverse} \cdot \text{reverse}$

$[a] \rightarrow [a]$

(i)  $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$[\text{Int}] \rightarrow [\text{Int}]$

(j)  $\text{let } f\ x = x \text{ in } (f\ 'a', f\ \text{True})$

$(a \rightarrow a, a \rightarrow a)$

(k)  $\text{fmap } (\text{fmap } (+1)) [\text{Nothing}]$

ill typed

$\text{fmap } (+1) :$

### Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) Int -> Int

*Example answers:*

\x -> x + 1

(+1)

(b) Bool -> [Bool]

$\lambda b \rightarrow \text{if } b \text{ then } [b]$

(c) a -> Maybe b

*undefined*

(d) (Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]

*zipWith (==) [1..5] ['a','b']*

(e) (a -> b -> c) -> Maybe a -> Maybe b -> Maybe c

$\lambda f \ a \ b \rightarrow \text{do } a \leftarrow a \ b \leftarrow b \ \text{return } f \ a \ b$

(f) (a -> b) -> (b -> Bool) -> [a] -> [b]

$\lambda f \ \_ \ f1 \ f2 \ i \rightarrow \text{map } (f2.f1) \ i$

(g) Maybe a -> (a -> [b]) -> [(a,b)]

$\lambda 'ma \rightarrow$

(h) Eq a => a -> [a] -> [a]

*delete*

(i) Show a => [a] -> IO String

$\lambda xs \rightarrow \text{putStrLn } (\text{map } (\text{show}) \ xs)$

(j) (a,b) -> (a -> b -> c) -> c

$\lambda (a,b) \ f \rightarrow f \ a \ b$

(k) ((a,b) -> c) -> c

*undefined*

### Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```

method Cube(m:int) returns (y:int)
 requires m > 0
 ensures y == m*m*m
{
 y := 0;
 var x := m * m;
 var z := m;
 while (z > 0)
 {
 z := z - 1;
 y := y + x;
 }
}

```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post-conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with  $\rightarrow$ . These implication steps can (silently) rely on all the usual rules of arithmetic.

```

{ { m > 0 } } $\rightarrow$
{ { m * m = m * m } } 0 = (m - m) * m * m { { m ≥ 0 } }
 y := 0;
 { { m * m = m * m } } y = (m - m) * m * m { { m ≥ 0 } }
 var x := m * m;
 { { x = m * m } } y = (m - m) * x * { { m ≥ 0 } }
 var z := m;
 { { x = m * m } } y = (m - z) * x * { { z ≥ 0 } }
 while (z > 0) {
 { { x = m * m } } y = (m - z) * x * { { z ≥ 0 } } { { z > 0 } } $\rightarrow$
 { { x = m * m } } y + x = (m - (z - 1)) * x * { { z - 1 ≥ 0 } }
 z := z - 1;
 { { x = m * m } } y + x = (m - z) * x * { { z ≥ 0 } }
 y := y + x;
 { { x = m * m } } y = (m - z) * x * { { z ≥ 0 } }
 { { x = m * m } } y = (m - z) * x * { { z > 0 } } $\rightarrow$
 { { y = m * m } }

```



### Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave [] [] = []`

`weave (x:xs) (y:ys) = (x:y:weave xs ys)`

- (b) Implement a function `powerSet`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerSet [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`powerSet [] = [[]]`

`powerSet (x:xs) = foldl (\acc y -> y:acc`

`powerSet xs = foldl (\acc x -> foldl (\acc' x' ->`  
`[x, x':acc'])`





## CMSC 433: Midterm Exam (Spring 2024)

## Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

$$a_1, a_2, a_3, a_4, \dots, \sum_{i=1}^n (-1)^{i-1} a_i$$

$$[a_1, a_2, a_3, a_4] : a_1 = \left( \begin{matrix} 1 \\ 0 \end{matrix} \right), (-1)^0 [a_2, \dots, a_4] = a_1 - \left( \begin{matrix} 1 \\ 0 \end{matrix} \right) a_2 = \left( \begin{matrix} 1 \\ 0 \end{matrix} \right) a_2 - \left( \begin{matrix} 1 \\ 0 \end{matrix} \right) a_3 = \left( \begin{matrix} 1 \\ 0 \end{matrix} \right) a_3 - \left( \begin{matrix} 1 \\ 0 \end{matrix} \right) a_4 = \left( \begin{matrix} 1 \\ 0 \end{matrix} \right) a_4$$

$$0 - a_1 - a_2 - a_3 - \dots - a_n \quad \text{foldl } (-) \text{ } (-a_1) [a_2, \dots, a_n] = \text{foldl } (-) \text{ } (-a_1 - a_2) [a_3, \dots, a_n]$$

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

|                                                                                                                                                                                                                                      |                                                                                                                                                                               |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>method f1(a:array&lt;int&gt;) returns (out:int) {     out := 0;     var sign := 1;     var i := 0;     while (i &lt; a.length) {         out := out + sign * a[i];         sign := sign * (-1);         i := i+1;     } }</pre> | <pre>method f2(a:array&lt;int&gt;) returns (out:int) {     out := 0;     var i := 0;     while (i &lt; a.length) {         out := out - a[i];         i := i+1;     } }</pre> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or “ill-typed” if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a)  $\backslash x \rightarrow x$

*Example answer:*

$a \rightarrow a$

(b)  $\backslash x\ y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c)  $\backslash x\ y \rightarrow$  if  $x == y$  then show  $x$  else show  $(x,y)$

$\text{Eq } a \Rightarrow a \rightarrow a \rightarrow \text{String}$

(d)  $\backslash x\ l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e)  $\text{foldr} (\text{const } 1)$

*ill-typed*

(f)  $(\text{"42"})$

$a \rightarrow (a, \text{String})$

(g)  $(\text{"42"})$

$a \rightarrow (\text{String}, a)$

(h)  $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i)  $\backslash l \rightarrow \text{filter} (< 42) (l ++ l)$

$(\text{Num } a, \text{Ord } a) \Rightarrow [a] \rightarrow [a]$

(j)  $\text{let } f\ x = x \text{ in } (f\ 'a', f\ \text{True})$

$(\text{Ord } a, \text{Eq } a)$

(k)  $\text{fmap} (\text{fmap} (+1)) [\text{Nothing}]$

*ill-typed*

$\therefore a \rightarrow b \rightarrow (a,b)$

### Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

*Example answers:*

`\x -> x + 1`

`(+1)`

(b) `Bool -> [Bool]`

`\x -> x : [x]`

(c) `a -> Maybe b`

`undefined`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`zipWith (\x y -> x1 == 0 && y == 'c') xs ys`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`\f x y -> fmap f x <> y`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`\f p xs -> filter p (map f xs)`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

`\x f -> zip (maybeToList x) (concat $ maybeToList $ fmap f x)`

(h) `Eq a => a -> [a] -> [a]`

`\x xs -> (concat . group) (x:xs)`

(i) `Show a => [a] -> IO String`

`putStrLn $ concat . map show`

(j) `(a,b) -> (a -> b -> c) -> c`

`\(x,y) f -> f x y`

(k) `((a,b) -> c) -> c`

`undefined`

### Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
 requires m > 0
 ensures y == m*m*m
{
 y := 0;
 var x := m * m;
 var z := m;
 while (z > 0)
 {
 z := z - 1;
 y := y + x;
 }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post-conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules - write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with  $\rightarrow>$ . These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } } $\rightarrow>$
{ { m ≥ 0
 y := 0;
 if m ≥ 0
 {
 var x := m * m;
 var z := m;
 while (z > 0) {
 { { y == (m-z)*x } } $\rightarrow>$
 { { y + x == (m-z+1)*x } }
 z := z - 1;
 { { y + x == (m-z)*x } }
 y := y + x;
 { { y == (m-z)*x } }
 }
 { { y == (m-z)*x } } $\rightarrow>$
 { { y == m * m } }
```

### Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
weave xs ys = concat (zipWith (:) xs ys)
```

- (b) Implement a function `powerSet`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerSet [1,2,3] = [[], [1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]
powerSet [] = [[]]
powerSet [x:xs] = let p = powerSet xs in p ++ map (x:) p
```





Yoseph Tezera  
118042626

## CMSC 433: Midterm Exam (Spring 2024)

### Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

|                                                                                                                                                                                        |                                                                                                                                                                                       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>method f1(a:array&lt;int&gt;) returns (out:int) {   var i := a.Length-1;    while (i &gt; -1)     invariant 0 ≤ i ≤ a.Length   {     out := out - a[i];     i := i-1;   } }</pre> | <pre>method f2(a:array&lt;int&gt;) returns (out:int) {   var i := 0;    while (i &lt; a.Length-1)     invariant 0 ≤ i ≤ a.Length   {     out := out - a[i];     i := i+1;   } }</pre> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

$x :: []$

## Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a)  $\lambda x \rightarrow x$

*Example answer:*

$a \rightarrow a$

(b)  $\lambda x y \rightarrow (x, y)$

$a \rightarrow b \rightarrow (a, b)$

(c)  $\lambda x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x, y)$

$a \rightarrow b \rightarrow \text{String}$

(d)  $\lambda x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e)  $\text{foldr } (\text{const } 1)$

$b \rightarrow (a \rightarrow b \rightarrow a) \rightarrow b$

(f)  $(\cdot, 42)$

$a \rightarrow (a, \text{String})$

(g)  $(\cdot) "42"$

$a \rightarrow (\text{String}, a)$

(h)  $\text{reverse} \cdot \text{reverse}$

$[a] \rightarrow [a] \rightarrow ([b] \rightarrow [b]) \rightarrow b \rightarrow a$

(i)  $\lambda l \rightarrow \text{filter } (< 42) (l ++ l)$

$[Int] \rightarrow [Int]$

(j)  $\text{let } f \ x = x \text{ in } (f 'a', f \text{ True})$

$(a \rightarrow b) \rightarrow b$

(k)  $\text{fmap } (\text{fmap } (+1)) [\text{Nothing}]$

$(Int \rightarrow Int) \rightarrow [Maybe]$

$f :: (Int \rightarrow Int) \rightarrow [Maybe]$

$(Int \rightarrow Int) \rightarrow Int \rightarrow Int$



### Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

*Example answers:*

`\x -> x + 1`

`(+1)`

(b) `Bool -> [Bool]`

`\x -> if x then [x] else [not x]`

(c) `a -> Maybe b`

`foo a = Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`func f x:xs y:ys = if (f x y) then [] else (f x y):[True,False]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`func f a b = case (a,b) of (Just x, Just y) -> Just (f a b)`

`(_,_) -> Nothing`

`foo f g lst = if (g f head lst) then (f head lst) else []`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

`foo m f = case m of`  
`Just x -> let y = f x m [(x, head y)]`  
`Nothing -> []`

(h) `Eq a => a -> [a] -> [a]`

`foo a lst = if a == not(not a) then [a] else []`

(i) `Show a => [a] -> IO String`

`foo a = putStrLn (show a:[])`

(j) `(a,b) -> (a -> b -> c) -> c`

`foo (a,b) f = f a b`

(k) `((a,b) -> c) -> c`

`foo f = f (1,a)`

`f g`

### Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
 requires m > 0
 ensures y == m*m*m
{
 y := 0;
 var x := m * m;
 var z := m;
 while (z > 0)
 {
 z := z - 1;
 y := y + x;
 }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with  $\vdash \gg$ . These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } } $\vdash \gg$
{ { $y = m \cdot m \cdot m$ } } $\wedge$ $\{ \{ z \geq 0 \} \}$
 y := 0;
 { { $y = m \cdot m \cdot m$ } } $\wedge$ $\{ \{ y \geq 0 \}$
 var x := m * m;
 { { $y = m \cdot x$ } } $\wedge$ $\{ \{ y \geq 0 \}$
 var z := m;
 { { $y = z \cdot x$ } } $\wedge$ $\{ \{ y \geq 0 \}$
 while (z > 0) {
 { {
 y + x = (z - 1) * x $\wedge$ $\{ \{ y \geq 0 \}$
 z := z - 1;
 { { y + x = z * x } } $\wedge$ $\{ \{ y + x \geq 0 \}$
 y := y + x;
 { { y = z * x } } $\wedge$ $\{ \{ y \geq 0 \}$
 }
 { { $y(z \geq 0)$ } } $\wedge$ $\{ \{ y = z \cdot x$ } $\wedge$ $\{ \{ y \geq 0 \}$
 }
 { { y = m * m * m } }
```

inv = 0  $y = z * x$   $\wedge$   $y \geq 0$

## Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

where  $a \ b = \text{case } (a, b) \text{ of}$   
 $(x:x1:xs, y:y1:ys) \rightarrow x:y:x1:y1:(\text{weave } xs \ ys)$   
 $([x], [y]) \rightarrow x:y:[]$   
 $(-, -) \rightarrow []$

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[], [1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`powerset [] = case list of`

`[]  $\rightarrow$  []`

`[x]  $\rightarrow$  [x]:[]:[]`

`x:xs  $\rightarrow$  (aux x):(powerset xs) where`

`aux x = x:(head xs):(tail xs):[]`

`-  $\rightarrow$  []`



## Typeclass Definitions

```
class Semigroup a where
 (<>) :: a -> a -> a

class Semigroup m => Monoid m where
 mempty :: m

class Show a where
 show :: a -> String

class Eq a where
 (==) :: a -> a -> Bool
 (/=) :: a -> a -> Bool

class Eq a => Ord a where
 compare :: a -> a -> Ordering
 (<), (<=), (>=), (>) :: a -> a -> Bool
 max, min :: a -> a -> a

class Functor f where
 fmap :: (a -> b) -> f a -> f b

class Functor f => Applicative f where
 pure :: a -> f a
 (<*>) :: f (a -> b) -> f a -> f b

class Applicative m => Monad m where
 return :: a -> m a
 (>>=) :: m a -> (a -> m b) -> m b

class Foldable t where
 foldMap :: Monoid m => (a -> m) -> t a -> m

class Arbitrary a where
 arbitrary :: Gen a
 shrink :: a -> [a]

class Num a where
 (+), (-), (*) :: a -> a -> a
 negate :: a -> a
 abs :: a -> a
 signum :: a -> a
 fromInteger :: Integer -> a
```

## Prelude Functions and Datatypes

### Booleans

```
data Bool = False | True
```

```
(&&) :: Bool -> Bool -> Bool
(||) :: Bool -> Bool -> Bool
not :: Bool -> Bool
```

### Lists

```
data [] a = [] | a : [a]
```

```
(++) :: [a] -> [a] -> [a]
head, last :: [a] -> a
tail, init :: [a] -> [a]
```

```
null :: Foldable t => t a -> Bool
length :: Foldable t => t a -> Int
map :: (a -> b) -> [a] -> [b]
reverse :: [a] -> [a]
intersperse :: a -> [a] -> [a]
transpose :: [[a]] -> [[a]]
foldl :: Foldable t => (b -> a -> b) -> b -> t a -> b
foldr :: Foldable t => (a -> b -> b) -> b -> t a -> b
concat :: Foldable t => t [a] -> [a]
and, or :: Foldable t => t Bool -> Bool
any, all :: Foldable t => (a -> Bool) -> t a -> Bool
sum, product :: (Foldable t, Num a) => t a -> a
maximum, minimum :: (Foldable t, Ord a) => t a -> a
repeat :: a -> [a]
replicate :: Int -> a -> [a]
take, drop :: Int -> [a] -> [a]
splitAt :: Int -> [a] -> ([a], [a])
takeWhile, dropWhile :: (a -> Bool) -> [a] -> [a]
group :: Eq a => [a] -> [[a]]
inits, tails :: [a] -> [[a]]
elem, notElem :: (Foldable t, Eq a) => a -> t a -> Bool
lookup :: Eq a => a -> [(a, b)] -> Maybe b
find :: Foldable t => (a -> Bool) -> t a -> Maybe a
filter :: (a -> Bool) -> [a] -> [a]
zip :: [a] -> [b] -> [(a, b)]
zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
nub :: Eq a => [a] -> [a]
delete :: Eq a => a -> [a] -> [a]
sort :: Ord a => [a] -> [a]
sortOn :: Ord b => (a -> b) -> [a] -> [a]
```

### Maybe

```
data Maybe a = Nothing | Just a
```

```
maybe :: b -> (a -> b) -> Maybe a -> b
isJust, isNothing :: Maybe a -> Bool
listToMaybe :: [a] -> Maybe a
maybeToList :: Maybe a -> [a]
catMaybes :: [Maybe a] -> [a]
mapMaybe :: (a -> Maybe b) -> [a] -> [b]
```



## 1 Hoare Rules

$$\frac{\{P\} s1 \{Q\} \quad \{Q\} s2 \{R\}}{\{P\} s1; s2 \{R\}} \text{ (hoare\_seq)}$$

$$\frac{}{\{Q\} [x := a] \{x := a \wedge Q\}} \text{ (hoare\_asgn)}$$

$$\frac{\{P'\} c \{Q'\} \quad P \Rightarrow P' \quad Q' \Rightarrow Q}{\{P\} c \{Q\}} \text{ (hoare\_consequence)}$$

$$\frac{\{P \wedge \wedge b\} s1 \{Q\} \quad \{P \wedge \wedge !b\} s2 \{Q\}}{\{P\} \text{ if } b \{ s1 \} \text{ else } \{ s2 \} \{Q\}} \text{ (hoare\_if)}$$

$$\frac{\{P \wedge \wedge b\} s \{P\}}{\{P\} \text{ while } b \{ s \} \{P \wedge \wedge !b\}} \text{ (hoare\_while)}$$

## Functors, Applicatives, and Monads

```
(<$> :: Functor f => (a -> b) -> f a -> f b
(<$) :: Functor f => a -> f b -> f a
($>) :: Functor f => f a -> b -> f b

liftA2 :: Applicative f => (a -> b -> c) -> f a -> f b -> f c
(*) :: Applicative f => f a -> f b -> f b
(<*) :: Applicative f => f a -> f b -> f a
when :: Applicative f => Bool -> f () -> f ()

(>>) :: Monad m => m a -> m b -> m b
mapM :: Monad m => (a -> m b) -> [a] -> m [b]
filterM :: Monad m => (a -> m Bool) -> [a] -> m [a]
sequence :: Monad m => [m a] -> m [a]
join :: Monad m => m (m a) -> m a
zipWithM :: Monad m => (a -> b -> m c) -> [a] -> [b] -> m [c]
foldM :: (Foldable t, Monad m) => (b -> a -> m b) -> b -> t a -> m b
replicateM :: Applicative m => Int -> m a -> m [a]

liftM :: Monad m => (a1 -> r) -> m a1 -> m r
liftM2 :: Monad m => (a1 -> a2 -> r) -> m a1 -> m a2 -> m r
...

```



# CMSC 433: Midterm Exam (Spring 2024)

Seunghyun Sung

## Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldl (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

|                                                                                                                                                                    |                                                                                                                                                                   |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>method f1(a:array&lt;int&gt;) returns (out:int) {   var i:int := a.length;   out := 0;   while i &gt;= 0 {     out := out - a[i];     i := i - 1;   } }</pre> | <pre>method f2(a:array&lt;int&gt;) returns (out:int) {   var i:int := 0;   out := 0;   while i &lt; a.length {     out := out - a[i];     i := i + 1;   } }</pre> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a)  $\lambda x \rightarrow x$

*Example answer:*

$a \rightarrow a$

(b)  $\lambda x y \rightarrow (x, y)$

$a \rightarrow b \rightarrow (a, b)$

(c)  $\lambda x y \rightarrow$  if  $x == y$  then show  $x$  else show  $(x, y)$

$a \rightarrow a \rightarrow \text{String}$

(d)  $\lambda x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e)  $\text{foldr } (\text{const } 1)$

$\text{Foldable } t \Rightarrow b \rightarrow t a \rightarrow b$

(f)  $(\text{"42"})$

ill-typed

(g)  $(\text{"42"})$

$a \rightarrow (\text{String}, a)$

(h)  $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i)  $\lambda l \rightarrow \text{filter } (< 42) (l ++ l)$

$[a] \rightarrow [a]$

(j)  $\text{let } f x = x \text{ in } (f 'a', f \text{True})$

$(\text{Char}, \text{Bool})$

(k)  $\text{fmap } (\text{fmap } (+1)) [\text{Nothing}]$

$[\text{Maybe } a]$

foldr f zero list

### Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a)  $\text{Int} \rightarrow \text{Int}$

*Example answers:*

$\backslash x \rightarrow x + 1$

$(+1)$

(b)  $\text{Bool} \rightarrow [\text{Bool}]$

$\backslash x \rightarrow \text{if } x \text{ then } [\text{True}] \text{ else } [\text{False}]$

(c)  $a \rightarrow \text{Maybe } b$

$\backslash a \rightarrow \text{Nothing}$

(d)  $(\text{Int} \rightarrow \text{Char} \rightarrow \text{Bool}) \rightarrow [\text{Int}] \rightarrow [\text{Char}] \rightarrow [\text{Bool}]$

$\text{zipWith}$

(e)  $(a \rightarrow b \rightarrow c) \rightarrow \text{Maybe } a \rightarrow \text{Maybe } b \rightarrow \text{Maybe } c$

*undefined*

(f)  $(a \rightarrow b) \rightarrow (b \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [b]$

*undefined*

(g)  $\text{Maybe } a \rightarrow (a \rightarrow [b]) \rightarrow [(a.b)]$

$\backslash x \rightarrow \text{if } x == \text{Just}$

(h)  $\text{Eq } a \Rightarrow a \rightarrow [a] \rightarrow [a]$

$\backslash x \rightarrow \text{let } \rightarrow \text{if } x == x \text{ then } x : \text{let else let}$

(i)  $\text{Show } a \Rightarrow [a] \rightarrow \text{IO String}$

$\text{putStrLn} = \text{putStrLn}$

(j)  $(a.b) \rightarrow (a \rightarrow b \rightarrow c) \rightarrow c$

$\backslash (a,b), \text{func} \rightarrow \text{func } a \ b$

(k)  $((a.b) \rightarrow c) \rightarrow c$

*undefined*

### Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
```

```
 requires m > 0
```

```
 ensures y == m*m*m
```

```
{
```

```
 y := 0;
```

```
 var x := m * m;
```

```
 var z := m;
```

```
 while (z > 0)
```

```
 {
```

```
 z := z - 1;
```

```
 y := y + x;
```

```
 }
```

```
}
```

Handwritten annotations for the program above:

- Before `y := 0;`:  $y = 0$
- Before `var x := m * m;`:  $x = m \cdot m$
- Before `var z := m;`:  $z = m$
- Before `while (z > 0)`:  $m \cdot m \cdot m - 1$
- Inside the while loop, before `z := z - 1;`:  $m \cdot m \cdot 2(m \cdot m) m - 2$
- Inside the while loop, before `y := y + x;`:  $m \cdot m \cdot 3(m \cdot m) m - 3$
- After the while loop:  $m \cdot m \cdot m(m \cdot m)$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post-conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with  $\rightarrow$ . These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } } $\rightarrow$
{ { m > 0 } } $\rightarrow$
 y := 0;
{ { m > 0 } } $\rightarrow$
 var x := m * m;
{ { m > 0 } } $\rightarrow$
 var z := m;
 while (z > 0) {
 { { z > 0 } } $\rightarrow$
 y := x * (m - z);
 { { z > 0 } } $\rightarrow$
 z := z - 1;
 { { z > 0 } } $\rightarrow$
 y := y + x;
 { { z > 0 } } $\rightarrow$
 y := x * (m - z);
 }
 { { z > 0 } } $\rightarrow$
 { { y = m * m * m } }
```

Handwritten annotations for the program above:

- Before `y := 0;`:  $y = 0$
- Before `var x := m * m;`:  $x = m \cdot m$
- Before `var z := m;`:  $z = m$
- Before `while (z > 0)`:  $m \cdot m \cdot m - 1$
- Inside the while loop, before `z := z - 1;`:  $m \cdot m \cdot 2(m \cdot m) m - 2$
- Inside the while loop, before `y := y + x;`:  $m \cdot m \cdot 3(m \cdot m) m - 3$
- After the while loop:  $m \cdot m \cdot m(m \cdot m)$

Handwritten annotations for the program above:

- Before `y := 0;`:  $y = 0$
- Before `var x := m * m;`:  $x = m \cdot m$
- Before `var z := m;`:  $z = m$
- Before `while (z > 0)`:  $m \cdot m \cdot m - 1$
- Inside the while loop, before `z := z - 1;`:  $m \cdot m \cdot 2(m \cdot m) m - 2$
- Inside the while loop, before `y := y + x;`:  $m \cdot m \cdot 3(m \cdot m) m - 3$
- After the while loop:  $m \cdot m \cdot m(m \cdot m)$

Handwritten annotations for the program above:

- Before `y := 0;`:  $y = 0$
- Before `var x := m * m;`:  $x = m \cdot m$
- Before `var z := m;`:  $z = m$
- Before `while (z > 0)`:  $m \cdot m \cdot m - 1$
- Inside the while loop, before `z := z - 1;`:  $m \cdot m \cdot 2(m \cdot m) m - 2$
- Inside the while loop, before `y := y + x;`:  $m \cdot m \cdot 3(m \cdot m) m - 3$
- After the while loop:  $m \cdot m \cdot m(m \cdot m)$

Handwritten annotations for the program above:

- Before `y := 0;`:  $y = 0$
- Before `var x := m * m;`:  $x = m \cdot m$
- Before `var z := m;`:  $z = m$
- Before `while (z > 0)`:  $m \cdot m \cdot m - 1$
- Inside the while loop, before `z := z - 1;`:  $m \cdot m \cdot 2(m \cdot m) m - 2$
- Inside the while loop, before `y := y + x;`:  $m \cdot m \cdot 3(m \cdot m) m - 3$
- After the while loop:  $m \cdot m \cdot m(m \cdot m)$

### Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave [] [] = []`

`weave (x:xs) (y:ys) = x:y:(weave xs ys)`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[], [1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`powerset [] = []`

`powerset (x:xs) = [x] ++ xs ++ x:xs ++ []`

`powerset (x:xs:xs) = [x] ++ [xs] ++`

`powerset (x:xs) ++ powerset (xs:xs)`  
`++ powerset (xs)`

`[1,2] [1,2,1,2]`

`[] [2,3] [2] [3]`  
`x xs`

`x:xs:xs`

`x:xs`

`1 2 3`  
`[1,2,1,2]`

`[1] [2] [3]`

`[1,2]`

`5 [1,2] [2,3]`

`[3,2,1]`





Cameron Coleman

## CMSC 433: Midterm Exam (Spring 2024)

### Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

|                                                                                                                                                                  |                                                                                                                                                               |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>method f1(a:array&lt;int&gt;) returns (out:int) {   var i := a.Length-1;   out := 0;   while (i &gt; -1) {     out = out - a[i];     i = i - 1;   } }</pre> | <pre>method f2(a:array&lt;int&gt;) returns (out:int) {   var i := 0;   out := 0;   while (i &lt; a.Length) {     out = out - a[i];     i = i + 1;   } }</pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|





### Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the `appendix`; do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

*Example answers:*

`\x -> x + 1`  
`(+1)`

(b) `Bool -> [Bool]`

`\x -> if x then [x] else [x,x]`

(c) `a -> Maybe b`

`\x -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`\f (i::is) (c:cs) -> if f (i+1) (c:"hi") then [True, False] else [False, True]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`\x (Just a) (Just b) -> Just (x a b)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`\x f (a:as) -> if f (x a) then [x a] else [f x a]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

`\(Just x) f -> [x] ++ [f x]`

(h) `Eq a => a -> [a] -> [a]`

`\x (y:ys) -> if x==y then (x:ys) else ys deriving (Eq)`

(i) `Show a => [a] -> IO String`

`\(x:xs) -> putStr x deriving (Show)`

(j) `(a,b) -> (a -> b -> c) -> c`

`\(x,y) f -> f x y`

(k) `((a,b) -> c) -> c`

`f (1,"2")`

### Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
 requires m > 0
 ensures y == m*m*m
{
 y := 0;
 var x := m * m;
 var z := m;
 while (z > 0)
 {
 z := z - 1;
 y := y + x;
 }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules – write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with  $\rightarrow>$ . These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } } $\rightarrow>$
{ { 0 == (m-m) * (m * m) } }
 y := 0;
 { { y == (m-m) * (m * m) } } m > 0
 var x := m * m;
 { { y == (m-m) * x } } m > 0
 var z := m;
 { { y == (m-z) * x } } z == m m > 0
 while (z > 0) {
 { { z > 0 } } { { y == (m-z) * x } } m > 0 $\rightarrow>$
 { { y + x == (m-z-1) * x } } m > 0
 z := z - 1;
 { { y + x == (m-z) * x } } m > 0
 y := y + x;
 { { y == (m-z) * x } } m > 0
 }
 { { (z > 0) } } { { y == (m-z) * x } } m > 0
 }
 { { y == m * m * m } }
 }
 }
 }
}
```

### Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave (x:xs) (y:ys) = [x,y] ++ (weave xs ys)
weave [] [] = []
```

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[], [1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]
```

```
powerset [] = [total (aux) []] where
```

```
aux val acc = [val] : acc
```



# Prelude Functions and Datatypes

## Booleans

```
data Bool = False | True

(&&) :: Bool -> Bool -> Bool
(||) :: Bool -> Bool -> Bool
not :: Bool -> Bool
```

## Lists

```
data [] a = [] | a : [a]

(++) :: [a] -> [a] -> [a]
head, last :: [a] -> a
tail, init :: [a] -> [a]

null :: Foldable t => t a -> Bool
length :: Foldable t => t a -> Int
map :: (a -> b) -> [a] -> [b]
reverse :: [a] -> [a]
intersperse :: a -> [a] -> [a]
transpose :: [[a]] -> [[a]]
foldl :: Foldable t => (b -> a -> b) -> b -> t a -> b
foldr :: Foldable t => (a -> b -> b) -> b -> t a -> b
concat :: Foldable t => t [a] -> [a]
and, or :: Foldable t => t Bool -> Bool
any, all :: Foldable t => (a -> Bool) -> t a -> Bool
sum, product :: (Foldable t, Num a) => t a -> a
maximum, minimum :: (Foldable t, Ord a) => t a -> a
repeat :: a -> [a]
replicate :: Int -> a -> [a]
take, drop :: Int -> [a] -> [a]
splitAt :: Int -> [a] -> ([a], [a])
takeWhile, dropWhile :: (a -> Bool) -> [a] -> [a]
group :: Eq a => [a] -> [[a]]
inits, tails :: [a] -> [[a]]
elem, notElem :: (Foldable t, Eq a) => a -> t a -> Bool
lookup :: Eq a => a -> [(a, b)] -> Maybe b
find :: Foldable t => (a -> Bool) -> t a -> Maybe a
filter :: (a -> Bool) -> [a] -> [a]
zip :: [a] -> [b] -> [(a, b)]
zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
nub :: Eq a => [a] -> [a]
delete :: Eq a => a -> [a] -> [a]
sort :: Ord a => [a] -> [a]
sortOn :: Ord b => (a -> b) -> [a] -> [a]
```

## Maybe

```
data Maybe a = Nothing | Just a

maybe :: b -> (a -> b) -> Maybe a -> b
isJust, isNothing :: Maybe a -> Bool
listToMaybe :: [a] -> Maybe a
maybeToList :: Maybe a -> [a]
catMaybes :: [Maybe a] -> [a]
mapMaybe :: (a -> Maybe b) -> [a] -> [b]
```

## Typeclass Definitions

```
class Semigroup a where
 (<>) :: a -> a -> a

class Semigroup m => Monoid m where
 mempty :: m

class Show a where
 show :: a -> String

class Eq a where
 (==) :: a -> a -> Bool
 (/=) :: a -> a -> Bool

class Eq a => Ord a where
 compare :: a -> a -> Ordering
 (<), (<=), (>=), (>) :: a -> a -> Bool
 max, min :: a -> a -> a

class Functor f where
 fmap :: (a -> b) -> f a -> f b

class Functor f => Applicative f where
 pure :: a -> f a
 (<*>) :: f (a -> b) -> f a -> f b

class Applicative m => Monad m where
 return :: a -> m a
 (>>=) :: m a -> (a -> m b) -> m b

class Foldable t where
 foldMap :: Monoid m => (a -> m) -> t a -> m

class Arbitrary a where
 arbitrary :: Gen a
 shrink :: a -> [a]

class Num a where
 (+), (-), (*) :: a -> a -> a
 negate :: a -> a
 abs :: a -> a
 signum :: a -> a
 fromInteger :: Integer -> a
```



## Functors, Applicatives, and Monads

```
(<$>) :: Functor f => (a -> b) -> f a -> f b
(<$) :: Functor f => a -> f b -> f a
($>) :: Functor f => f a -> b -> f b

liftA2 :: Applicative f => (a -> b -> c) -> f a -> f b -> f c
(*) :: Applicative f => f a -> f b -> f b
(<*) :: Applicative f => f a -> f b -> f a
when :: Applicative f => Bool -> f () -> f ()

(>>) :: Monad m => m a -> m b -> m b
mapM :: Monad m => (a -> m b) -> [a] -> m [b]
filterM :: Monad m => (a -> m Bool) -> [a] -> m [a]
sequence :: Monad m => [m a] -> m [a]
join :: Monad m => m (m a) -> m a
zipWithM :: Monad m => (a -> b -> m c) -> [a] -> [b] -> m [c]
foldM :: (Foldable t, Monad m) => (b -> a -> m b) -> b -> t a -> m b
replicateM :: Applicative m => Int -> m a -> m [a]

liftM :: Monad m => (a1 -> r) -> m a1 -> m r
liftM2 :: Monad m => (a1 -> a2 -> r) -> m a1 -> m a2 -> m r
```

## 1 Hoare Rules

$$\frac{\begin{array}{l} \{P\} s1 \{Q\} \\ \{Q\} s2 \{R\} \end{array}}{\{P\} s1; s2 \{R\}} \quad (\text{hoare\_seq})$$
$$\frac{}{\{Q \ [x := a]\} x := a \{Q\}} \quad (\text{hoare\_asgn})$$
$$\frac{\begin{array}{l} \{P'\} c \{Q'\} \\ P \implies P' \quad Q' \implies Q \end{array}}{\{P\} c \{Q\}} \quad (\text{hoare\_consequence})$$
$$\frac{\begin{array}{l} \{P \ \&\& \ b\} s1 \{Q\} \\ \{P \ \&\& \ !b\} s2 \{Q\} \end{array}}{\{P\} \text{if } b \{ s1 \} \text{else } \{ s2 \} \{Q\}} \quad (\text{hoare\_if})$$
$$\frac{\{P \ \&\& \ b\} s \{P\}}{\{P\} \text{while } b \{ s \} \{P \ \&\& \ !b\}} \quad (\text{hoare\_while})$$



## Prelude Functions and Datatypes

### Booleans

```
data Bool = False | True

(&&) :: Bool -> Bool -> Bool
(||) :: Bool -> Bool -> Bool
not :: Bool -> Bool
```

### Lists

```
data [] a = [] | a : [a]

(++) :: [a] -> [a] -> [a]
head, last :: [a] -> a
tail, init :: [a] -> [a]

null :: Foldable t => t a -> Bool
length :: Foldable t => t a -> Int
map :: (a -> b) -> [a] -> [b]
reverse :: [a] -> [a]
intersperse :: a -> [a] -> [a]
transpose :: [[a]] -> [[a]]
foldl :: Foldable t => (b -> a -> b) -> b -> t a -> b
foldr :: Foldable t => (a -> b -> b) -> b -> t a -> b
concat :: Foldable t => t [a] -> [a]
and, or :: Foldable t => t Bool -> Bool
any, all :: Foldable t => (a -> Bool) -> t a -> Bool
sum, product :: (Foldable t, Num a) => t a -> a
maximum, minimum :: (Foldable t, Ord a) => t a -> a
repeat :: a -> [a]
replicate :: Int -> a -> [a]
take, drop :: Int -> [a] -> [a]
splitAt :: Int -> [a] -> ([a], [a])
takeWhile, dropWhile :: (a -> Bool) -> [a] -> [a]
group :: Eq a => [a] -> [[a]]
inits, tails :: [a] -> [[a]]
elem, notElem :: (Foldable t, Eq a) => a -> t a -> Bool
lookup :: Eq a => a -> [(a, b)] -> Maybe b
find :: Foldable t => (a -> Bool) -> t a -> Maybe a
filter :: (a -> Bool) -> [a] -> [a]
zip :: [a] -> [b] -> [(a, b)]
zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
nub :: Eq a => [a] -> [a]
delete :: Eq a => a -> [a] -> [a]
sort :: Ord a => [a] -> [a]
sortOn :: Ord b => (a -> b) -> [a] -> [a]
```

### Maybe

```
data Maybe a = Nothing | Just a

maybe :: b -> (a -> b) -> Maybe a -> b
isJust, isNothing :: Maybe a -> Bool
listToMaybe :: [a] -> Maybe a
maybeToList :: Maybe a -> [a]
catMaybes :: [Maybe a] -> [a]
mapMaybe :: (a -> Maybe b) -> [a] -> [b]
```

## Typeclass Definitions

```
class Semigroup a where
 (<>) :: a -> a -> a

class Semigroup m => Monoid m where
 empty :: m

class Show a where
 show :: a -> String

class Eq a where
 (==) :: a -> a -> Bool
 (/=) :: a -> a -> Bool

class Eq a => Ord a where
 compare :: a -> a -> Ordering
 (<), (<=), (>=), (>) :: a -> a -> Bool
 max, min :: a -> a -> a

class Functor f where
 fmap :: (a -> b) -> f a -> f b

class Functor f => Applicative f where
 pure :: a -> f a
 (<*>) :: f (a -> b) -> f a -> f b

class Applicative m => Monad m where
 return :: a -> m a
 (>>=) :: m a -> (a -> m b) -> m b

class Foldable t where
 foldMap :: Monoid m => (a -> m) -> t a -> m

class Arbitrary a where
 arbitrary :: Gen a
 shrink :: a -> [a]

class Num a where
 (+), (-), (*) :: a -> a -> a
 negate :: a -> a
 abs :: a -> a
 signum :: a -> a
 fromInteger :: Integer -> a
```

## 1 Hoare Rules

$$\frac{\begin{array}{l} \{P\} s_1 \{Q\} \\ \{Q\} s_2 \{R\} \end{array}}{\{P\} s_1; s_2 \{R\}} \text{ (hoare\_seq)}$$

$$\frac{}{\{Q\} [x := a] \{x := a \wedge Q\}} \text{ (hoare\_asgn)}$$

$$\frac{\begin{array}{l} \{P'\} c \{Q'\} \\ P \Rightarrow P' \quad Q' \Rightarrow Q \end{array}}{\{P\} c \{Q\}} \text{ (hoare\_consequence)}$$

$$\frac{\begin{array}{l} \{P \wedge \wedge b\} s_1 \{Q\} \\ \{P \wedge \wedge !b\} s_2 \{Q\} \end{array}}{\{P\} \text{ if } b \{ s_1 \} \text{ else } \{ s_2 \} \{Q\}} \text{ (hoare\_if)}$$

$$\frac{\{P \wedge \wedge b\} s \{P\}}{\{P\} \text{ while } b \{ s \} \{P \wedge \wedge !b\}} \text{ (hoare\_while)}$$

## Functors, Applicatives, and Monads

```
(<$>) :: Functor f => (a -> b) -> f a -> f b
(<$) :: Functor f => a -> f b -> f a
($>) :: Functor f => f a -> b -> f b

liftA2 :: Applicative f => (a -> b -> c) -> f a -> f b -> f c
(*) :: Applicative f => f a -> f b -> f b
(<*) :: Applicative f => f a -> f b -> f a
when :: Applicative f => Bool -> f () -> f ()

(>>) :: Monad m => m a -> m b -> m b
mapM :: Monad m => (a -> m b) -> [a] -> m [b]
filterM :: Monad m => (a -> m Bool) -> [a] -> m [a]
sequence :: Monad m => [m a] -> m [a]
join :: Monad m => m (m a) -> m a
zipWithM :: Monad m => (a -> b -> m c) -> [a] -> [b] -> m [c]
foldM :: (Foldable t, Monad m) => (b -> a -> m b) -> b -> t a -> m b
replicateM :: Applicative m => Int -> m a -> m [a]

liftM :: Monad m => (a1 -> r) -> m a1 -> m r
liftM2 :: Monad m => (a1 -> a2 -> r) -> m a1 -> m a2 -> m r
...
```

## CMSC 433: Midterm Exam (Spring 2024)

## Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

*[1, 2, 3, 4, 5]*

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

*(-) 0 1*  
*(-) (-1) (-2)*  
*(-) 5 0*

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

|                                                                                                                                                                       |                                                                                                                                                                    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>method f1(a:array&lt;int&gt;) returns (out:int) {   out := 0;   var i: int := a.length-1;   while (i &gt;= 0) {     out := a[i] - out;     i := i-1;   } }</pre> | <pre>method f2(a:array&lt;int&gt;) returns (out:int) {   out := 0;   var i: int := 0;   while (i &lt; a.length) {     out := out - a[i];     i := i+1;   } }</pre> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|

```
}
}
```

## Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a)  $\lambda x \rightarrow x$

*Example answer:*

$a \rightarrow a$

(b)  $\lambda x y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c)  $\lambda x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

*E.g.  $a \Rightarrow a \rightarrow a \rightarrow \text{String}$*

(d)  $\lambda x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e)  $\text{foldr} (\text{const } 1)$

*ill-typed*

(f)  $(\text{"42"})$

$a \rightarrow (a, \text{String})$

(g)  $(\text{"42"})$

$a \rightarrow (\text{String}, a)$

(h)  $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i)  $\lambda l \rightarrow \text{filter } (< 42) (l ++ l)$

*None  $a \Rightarrow [a] \rightarrow [a]$*

(j)  $\text{let } f \ x = x \text{ in } (f \ 'a', f \ \text{True})$

*ill-typed  $(\text{Char}, \text{Bool})$*

(k)  $\text{fmap} (\text{fmap } (+1)) [\text{Nothing}]$

*ill-typed*



### Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

*Example answers:*

`\x -> x + 1`  
`(+1)`

(b) `Bool -> [Bool]`

`\x -> if x then [x] else [True]`

(c) `a -> Maybe b`

`\x -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`\x y z -> (zipWith x y ++ [1] z ++ ['a']) ++ [True]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`\x y z -> listToMaybe (zipWith x (maybeToList y) (maybeToList z))`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`\x y z -> filter y (map x z)`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

`\x y -> zip (maybeToList x) (concat (map y (maybeToList x)))`

(h) `Eq a => a -> [a] -> [a]`

`delete`

(i) `Show a => [a] -> IO String`

`\x -> do`  
`n ← read x`  
`putStrLn n`  
`getLine`

(j) `(a,b) -> (a -> b -> c) -> c`

`\(x,y) z -> z x y`

(k) `((a,b) -> c) -> c`

`undefined`

### Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
 requires m > 0
 ensures y == m*m*m
{
 y := 0;
 var x := m * m;
 var z := m;
 while (z > 0)
 {
 z := z - 1;
 y := y + x;
 }
}
```

$y = x + x + \dots + x$  (m times)

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules -- write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with  $\vdash \rightarrow \triangleright$ . These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } } $\rightarrow \triangleright$
{ { 0 = m * m * (m - m) } } $\vee$ { { m * m = m * m } } $\wedge$ { { m > 0 } }
 y := 0;
 { { y = m * m * (m - m) } } $\wedge$ { { m * m = m * m } } $\wedge$ { { m > 0 } }
 var x := m * m;
 { { y = x * (m - m) } } $\wedge$ { { x = m * m } } $\wedge$ { { m > 0 } }
 var z := m;
 { { y = x * (m - z) } } $\wedge$ { { x = m * m } } $\wedge$ { { z > 0 } }
 while (z > 0) {
 { { y = x * (m - z) } } $\wedge$ { { x = m * m } } $\wedge$ { { z > 0 } }
 { { y + x = x * (m - (z - 1)) } } $\wedge$ { { x = m * m } } $\wedge$ { { z > 0 } }
 z := z - 1;
 { { y + x = x * (m - z) } } $\wedge$ { { x = m * m } } $\wedge$ { { z > 0 } }
 y := y + x;
 { { y = x * (m - z) } } $\wedge$ { { x = m * m } } $\wedge$ { { z > 0 } }
 }
 { { (y) = x * (m - z) } } $\wedge$ { { x = m * m } } $\wedge$ { { z > 0 } } $\wedge$ { { ! (z > 0) } } } $\rightarrow \triangleright$
 { { y = m * m * m } } } $\rightarrow \triangleright$
```



Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave :: [a] -> [a] -> [a]`  
`weave [] [] = []`  
`weave (x:xs) (y:ys) = x:y : (weave xs ys)`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[], [2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`powerset :: [a] -> [[a]]`  
`powerset xs = foldr (\x acc -> (map (x:) acc) ++ acc) [] xs`

[ ]

[ 1 ]

[ 2, 3 ] [ 3 ] [ 3 ] [ 1 ]

[ 1, 2, 3 ] [ 2, 3 ] [ 1 ] [ 3 ] [ 1 ]



Brett Laver

## CMSC 433: Midterm Exam (Spring 2024)

### Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

|                                                                                                                                                                                     |                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>method f1(a:array&lt;int&gt;) returns (out:int) {     var i:int := a.Length;     out := 0;     while (i &gt; 0) {         out := out - a[i];         i := i - 1;     } }</pre> | <pre>method f2(a:array&lt;int&gt;) returns (out:int) {     var i:int := 0;     out := 0;     while (i &lt; a.Length) {         out := out - a[i];         i := i + 1;     } }</pre> |
| <pre>}</pre>                                                                                                                                                                        | <pre>}</pre>                                                                                                                                                                        |

### Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are **provided** in the appendix at the end.

(a)  $\lambda x \rightarrow x$

*Example answer:*

$a \rightarrow a$

(b)  $\lambda x y \rightarrow (x,y)$

$a \rightarrow a \rightarrow (a,a)$

(c)  $\lambda x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

$a \rightarrow a \rightarrow \text{string}$

(d)  $\lambda x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e)  $\text{foldr } (\text{const } 1)$

$a \rightarrow [b] \rightarrow b$

(f)  $(\# "42")$

$\text{string} \rightarrow \text{string}$

(g)  $(.) \text{ "42"}$

$(a,b) \rightarrow \text{string} \rightarrow \text{string}$

(h)  $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a] \rightarrow [[a]]$

(i)  $\lambda l \rightarrow \text{filter } (< 42) (l ++ l)$

$[a] \rightarrow [a]$

(j)  $\text{let } f \ x = x \text{ in } (f \ 'a', f \ \text{True})$

(k)  $\text{fmap } (\text{fmap } (+1)) [\text{Nothing}]$

$(f a \rightarrow f b) \rightarrow f a \rightarrow \text{Nothing}$

### Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix; do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

*Example answers:*

`\x -> x + 1`

`(+1)`

(b) `Bool -> [Bool]`

`\x -> [True]`

(c) `a -> Maybe b`

`\x -> Just x`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`foo f x y = f [x+1] [y]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`foo f x y = f (Just x) (Just y)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`foo f g x = f (g)`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

`foo x f -> [f (Just x)]`

(h) `Eq a => a -> [a] -> [a]`

`foo x -> compare [x] [x]`

(i) `Show a => [a] -> IO String`

(j) `(a,b) -> (a -> b -> c) -> c`

`foo f g = f (`

(k) `((a,b) -> c) -> c`

### Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
 requires m > 0
 ensures y == m*m*m
{
 y := 0;
 var x := m * m;
 var z := m;
 while (z > 0)
 {
 z := z - 1;
 y := y + x;
 }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules. write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with  $\rightarrow>$ . These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } } $\rightarrow>$
{ { $y = m \cdot m$ $y = m \cdot m$; $z = 1$ } } $m \geq 0$ } }
 y := 0;
 { { $y = m \cdot m$ $y = m \cdot m$; $z = 1$ } } $\&\&$ $m \geq 0$ } }
 var x := m * m;
 { { $y = m \cdot m$ $y = m \cdot x$; $z = 1$ } } $m \geq 0$ } }
 var z := m;
 { { $y = m \cdot z$ $y = m \cdot z$; $z = 1$ } } $z \geq 0$ } }
 while (z > 0) {
 { { $y = m \cdot z$ $y = m \cdot z$; $z = 1$ } } $z \geq 0$ } }
 { { $y + x = m \cdot (z - 1)$ $y + x = m \cdot (z - 1)$; $z = 1$ } } $z > 0$ } } $\rightarrow>$
 z := z - 1;
 { { $y + x = m \cdot z$ $y + x = m \cdot z$; $z = 1$ } } $z \geq 0$ } }
 y := y + x;
 { { $y = m \cdot z$ $y = m \cdot z$; $z = 1$ } } $z \geq 0$ } }
 }
 { { $y = m \cdot z$ $y = m \cdot z$; $z = 1$ } } $z \geq 0$ } } $\rightarrow>$
 { { y = m * m * m } }
```

Handwritten calculations:

|   |    |   |   |
|---|----|---|---|
| m | y  | x | z |
| 3 | 0  | 9 | 3 |
| 3 | 9  | 9 | 2 |
| 3 | 18 | 9 | 1 |
| 3 | 27 | 9 | 0 |

$18 = 18$



### Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave [] [] = []
weave (x:xs) (y:ys) = (x,y) : weave xs ys
```

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[], [1,2], [1,3], [1], [2,3], [2], [3], []]
```

```
powerset [] = [[]]
powerset (x:xs) = (x:xs) ++ (x:tail xs) ++ (x:last xs) ++
 powerset xs
```





a Chandragiri

118912622

## CMSC 433: Midterm Exam (Spring 2024)

### Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

|                                                                                                                                                                 |                                                                                                                                                                                    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>method f1(a:array&lt;int&gt;) returns (out:int) {   var i := 1;   out := 0;   while (i &lt; a.length) {     out := out - a[i];     i := i + 1;   } }</pre> | <pre>method f2(a:array&lt;int&gt;) returns (out:int) {   var i := a.length - 2;   out := a[a.length - 1];   while (i &gt;= 0) {     out := out - a[i];     i := i - 1;   } }</pre> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Please swap. meant f1 for foldl, f2 for foldr

## Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a)  $\backslash x \rightarrow x$

*Example answer:*

$a \rightarrow a$

(b)  $\backslash x y \rightarrow (x, y)$

$a \rightarrow b \rightarrow (a, b)$

(c)  $\backslash x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x, y)$

$a \rightarrow a \rightarrow \text{String}$

(d)  $\backslash x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e)  $\text{foldr} (\text{const } 1)$

$\text{int} \rightarrow [\text{int}] \rightarrow \text{int}$

(f)  $(\cdot^{42})$

ill typed

(g)  $(\cdot)^{42}$

ill typed

(h)  $\text{reverse} . \text{reverse}$

ill typed

(i)  $\backslash l \rightarrow \text{filter} (< 42) (l ++ l)$

$[\text{int}] \rightarrow [\text{int}]$

(j)  $\text{let } f \ x = x \text{ in } (f 'a', f \text{ True})$

$(\text{Char}, \text{Bool})$

~~(k)  $\text{fmap} (\text{fmap } (+1)) [\text{Nothing}]$~~

ill-typed

$\text{foldr } (a \rightarrow b \rightarrow b) \rightarrow b \rightarrow \text{int} \rightarrow \text{int}$   
 $(\text{const } 1)$   
 $\text{int} \rightarrow b \rightarrow \text{int}$

$\text{filter } (\text{int} \rightarrow \text{Bool}) \rightarrow [\text{int}] \rightarrow [\text{int}]$   
 $[a] \rightarrow [a]$

$\text{map } (a \rightarrow b) \rightarrow [a] \rightarrow [b]$   
 $\downarrow$   
 $[\text{Nothing}] \rightarrow$

$(a \rightarrow b) \rightarrow [a] \rightarrow [b]$

$(+1)$

$(\text{int} \rightarrow \text{int}) \rightarrow [\text{int}] \rightarrow [\text{int}]$

$\& [\text{int}] \rightarrow [\text{int}]$

### Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

*Example answers:*

`\x -> x + 1`

`(+1)`

(b) `Bool -> [Bool]`

`\b -> b:[]`

(c) `a -> Maybe b`

`\a -> if a then Just 1 else Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`elem c "abc"`

`myFunc f (n:ns) (c:cs) = if (f(n+1) c) && / then True:(myFunc f ns cs)`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`else False:(myFunc f ns cs)`

`unc f a b -> if (isJust a) && isJust b then`

`myFunc f [] == []`

`(f) (a -> b) -> (b -> Bool) -> [a] -> [b]`

`myFunc f [] = []`

`unc f g [a:as] = if (g(f a)) then (fa):(fFunc f g as)`

`else (fa):[]`

`unc a f = if (isJust a) then Ca, head(fa):[] else []`

(h) `Eq a => a -> [a] -> [a]`

`unc a (b:bs) = if (a == b) then (b:bs) else (a:[])`

(i) `Show a => [a] -> IO String`

`ifunc [] = show ""`

`unc (a:as) = show a`

(j) `(a,b) -> (a -> b -> c) -> c`

`unc tup f = f (fst tup) (fst tup)`

(k) `((a,b) -> c) -> c`

`struct`

### Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
 requires m > 0
 ensures y == m*m*m
{
 y := 0;
 var x := m * m;
 var z := m;
 while (z > 0)
 {
 z := z - 1;
 y := y + x;
 }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules – write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with  $\rightarrow$ . These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } } $\rightarrow$
{ { 0 = m * m (m-m) && m * m = m * m && m > 0 } }
 y := 0;
 { { y = m * m (m-m) && m * m == m * m && m > 0 } }
 var x := m * m;
 { { y = x (m-m) && x == m * m && m > 0 } }
 var z := m;
 { { y = x (m-z) && x == m * m && z > 0 } }
 while (z > 0) {
 { { y = x (m-z) && x == m * m && (z > 0) && (z > 0) } } $\rightarrow$
 { { y + x = x (m-z) && x == m * m && z - 1 > 0 } }
 z := z - 1;
 { { y + x = x (m-z) && x == m * m && z > 0 } }
 y := y + x;
 { { y = x (m-z) && x == m * m && z > 0 } }
 }
 { { y = x (m-z) && x == m * m && (z > 0) && ! (z > 0) } } $\rightarrow$
 { { y = m * m * m } }
```

invariants:  
 $y = x(m-z)$   
 $x = m * m$   
 $z > 0$

$y = x(m-z) \ \&\& \ z = 0$   
 $y = x(m)$   
 $x = m * m$   
 $y = m^3$

| x | y  | m | z |
|---|----|---|---|
| 9 | 0  | 3 | 3 |
|   | 9  |   | 2 |
|   | 18 |   | 1 |
|   | 27 |   | 0 |

$y = q(3-z)$   
 $y = x(m-z)$

$q(0) = 0$   
 $q(3-2) = 9$   
 $q(3-1)$

### Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave [] [] = []`

`weave [ ] (b:bs) = (b:bs)`

`weave (a:as) [ ] = (a:as)`

`weave (a:as) (b:bs) = a:b:(weave as bs)`

- (b) Implement a function `powerSet`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerSet [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`powerSet [ ] = [ [ ] ]`

`powerSet (a:as) = filter (\a -> elem a) (getSet a:as)`

where `getSet [ ] = [ [ ] ]`

`getSet (a:as) = [ a ] ++ tails (a:as) ++ heads (a:as)`

`++ ((a:as):[ [ ] ]) ++ getSet (as)`

full list a:as

each element of list ca:       

combos -> tails?

`[1,2,3]:[[2,3],[3]]` 12 1

tails: [2,3] in its -> [1,2]

3 2 1

[1,3]

h: +

1,2,3,4

123

12 23

12 23

23 34

12 23

